

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/363530245>

FluxNet: a physics-informed learning-based Riemann solver for transcritical flows with non-ideal thermodynamics

Preprint in SSRN Electronic Journal · August 2022

DOI: 10.2139/ssrn.4216629

CITATIONS

0

READS

379

2 authors:



Jeremy C.-H. Wang
University of Waterloo

22 PUBLICATIONS 51 CITATIONS

SEE PROFILE



Jean-Pierre Hickey
University of Waterloo

131 PUBLICATIONS 1,227 CITATIONS

SEE PROFILE

FluxNet: a physics-informed learning-based Riemann solver for transcritical flows with non-ideal thermodynamics

Jeremy C. H. Wang^{a,*}, Jean-Pierre Hickey^a

^a*Department of Mechanical and Mechatronics Engineering, University of Waterloo, 200 University Ave West, Waterloo, N2L 3G1, Ontario, Canada*

Abstract

Traditional Riemann solvers fall into two broad categories: exact solvers, which require multiple iterations to achieve high accuracy, and approximate linearized solvers, which achieve fast runtime at the expense of reduced accuracy. Here we explore learning-based Riemann solvers, termed FluxNets, as a new approach that balances accuracy and computational efficiency especially for transcritical and supercritical flow problems involving non-ideal thermodynamics. Various designs and training strategies are applied to fully connected neural networks. By comparing data-driven versus physics-informed loss functions, as well as neural networks of varying size, the results show that order-of-magnitude reductions in error compared to the Roe solver can be achieved with compact architectures. Numerical validation on 1D and 2D test cases reveals that a physics-informed FluxNet promotes smoothness, numerical stability, and generalizability, and is capable of producing entropy-stable results free of non-physical expansion shocks as compared to the Roe solver. Parallelization can be leveraged to accelerate inference such that the gains in accuracy are achieved at one quarter the runtime of exact solvers. The trade-off in accuracy versus efficiency may be justified in the case of non-ideal flows where even minor errors can result in spurious oscillations and instabilities.

Keywords: Riemann solver, machine learning, physics-informed neural networks, transcritical flow, computational fluid dynamics, non-ideal thermodynamics

*Corresponding author: jc9wang@uwaterloo.ca

1. Introduction

In recent years, machine learning (ML) has emerged as a potential tool to address unresolved challenges in fluid mechanics, including the discovery and development of physical models [1, 2, 3, 4], the acceleration of non-ideal thermophysical computations [5, 6, 7, 8, 9], flow control and optimization [10, 11, 12, 13], and increasingly, the solution of physical laws described by partial differential equations [14, 15, 16, 17].

Despite the explosive interest in ML, Brunton *et al.* [18] note that certain nuances unique to fluid mechanics have yet to be addressed. For example, few studies have examined how learning algorithms should contend with vast physical scales, sensitivity to noise, presence of latent variables, sharp flow features, or transient states. In particular, recent works are just beginning to push the boundary of neural networks for high-speed flows. Raissi *et al.* [19] proposed physics-informed neural networks for supervised learning tasks while abiding by physical laws specified as nonlinear partial differential equations. Mao *et al.* [20] examined physics-informed approaches for studying high-speed flow problems. Bezgin *et al.* [21] trained a convolutional neural network to estimate weights in a weighted essentially non-oscillatory (WENO) scheme for nonclassical undercompressive shock problems. Whereas typical applications of ML are concerned mainly with predictive performance and generalizability, the use of ML in physics must continue to uphold the principle of interpretability. Together, these considerations have motivated the use of ML not to replace numerical schemes entirely, but rather, to address specific well-defined issues within fluid mechanics.

Riemann solvers may be one area where ML can enhance accuracy at minimal increase in computational expense and without compromising the explainability of the broader numerics. Current approximate solvers used in Computational Fluid Dynamics (CFD) simulations are designed to render a non-iterative means to estimate the star-state conditions, albeit at the expense of accuracy. The star-state conditions together with the initial conditions are then used to solve the remaining states of the Riemann problem, and in turn, the intercell flux [22]. ML could provide a means to estimate star-state conditions at greater accuracy than traditional approximate solvers, but less time complexity than exact solvers. Moreover, physics enforces that every set of initial conditions corresponds to only one set of

star-state conditions.

Magiera *et al.* [23] were the first to develop a neural network that predicts conditions in the star-state region. They trained networks of 5 to 7 layers with 20 to 70 nodes per hidden layer with exponential linear unit activation functions, adding a so-called *constraint-resolving* layer that ensures the star-state predictions satisfy the Rankine-Hugoniot shock jump conditions [24] within a specified tolerance. They also tested a more general Constraint-Adapted Loss (CAL) method wherein the loss function penalizes deviation from the constraint but is not guaranteed to satisfy the loss exactly. The mean L_1 errors show good agreement on the order of $O(10^{-2})$ and they were able to resolve the shock front precisely, however a discussion of computational costs was not included. The study was also limited to perfect gases where the polytropic relation significantly simplifies the thermodynamic relationships and thus lessens the value of a more computationally intense approach.

This work was soon followed by Gyrya *et al.* [25] who used a two-layer network with 64 nodes per hidden layer and ReLU activation functions, also to predict star-state conditions of an ideal gas. No physical constraints were explicitly incorporated. Still, the network achieved root-mean squared errors of $O(10^{-2})$ for the star-state conditions. The authors also attempted to train their network to predict speeds of the left- and right-moving waves (which are highly nonlinear functions of the star-state conditions), yielding unacceptable errors $> 50\%$. Compared to Magiera *et al.* [23], the network by Gyrya *et al.* was ostensibly cheaper with similarly low star-state errors, but numerical results were prone to spurious discontinuities not reflected in the averaged error metrics associated with the learning curves. Most importantly, Gyrya *et al.* output 9 variables in the star-state region, of which many are redundant and can be calculated using standard equations in fluid mechanics. Thus, it is possible that their network added unwarranted computational overhead to produce these extra variables that may instead be computed more quickly and accurately by solving the appropriate fluids equations explicitly. In both the Magiera *et al.* [23] and Gyrya *et al.* [25] studies, there is limited analysis on network complexity and the implications for under- or overfitting. Additionally, it should be noted that the range of training and test data can affect the network size required to achieve a certain level of accuracy, and therefore the ideal network size is not universal.

Tangential studies have also considered other topics related to Riemann solver design. Fuks & Tchelepi [26] attempted to design a physics-informed neural network to predict fluxes given initial conditions in two-phase porous

media, but noted challenges when training networks to minimize nonlinear loss surfaces with discontinuities. Dieselhorst *et al.* [27] trained neural networks to accelerate the primitive-to-conservative conversions in relativistic hydrodynamics, for which there is no analytical closed-form solution much like in non-ideal thermodynamics. Though these studies did not relate to flows with non-ideal thermodynamics, they suggest potential alternate network architectures and the possible benefits they may provide to the overall numerical scheme.

As such, there remains an opportunity to develop a learning-based Riemann solver that preserves the requisite wave structure of the solution, achieves low errors when predicting variables required for flux calculations, is computationally efficient compared to state-of-the-art approximate solvers, is physically consistent, and delivers these capabilities for flows with non-ideal thermodynamics. In transcritical flows especially, more sophisticated Riemann solvers are generally needed to preserve entropy stability—there may be opportunities to take advantage of parallel matrix algorithms to render efficient neural network computations relative to traditional methods.

Here we train different multi-layer perceptron networks to predict star-state variables in the Riemann problem with non-ideal thermodynamics, which in turn can be used to compute the intercell flux for Godunov schemes. For the present purpose, neural networks used for flux computations are termed *FluxNets*. As the literature reveals, prior efforts to leverage ML for the Riemann problem have tended to assume perfect gas thermodynamics and have used physics-informed loss functions and arbitrary network dimensions without much justification. Given the growing interest in non-ideal flow problems, where flux and thermophysical computations can jointly account for approximately 80% of total runtime [5], there may be performance gains in adopting a learning-based Riemann solver for arbitrary equations of state. In this study, it is shown that a compact physics-informed FluxNet can be trained whose errors are order-of-magnitude less than with approximate solvers, and whose time complexity is $\sim 25\%$ that of exact solvers when matrix computations are parallelized. Virtually all FluxNet designs examined in this paper predict star-state conditions with 0.1 to 0.3% error during training, however the physics-informed approach ensures smooth generalizable predictions. Additionally, it is shown that a learning-based approach produces smaller, less biased distributions of error in the star-state variables compared to the Roe family of solvers.

This paper is organized in the following manner. §2 and 3 cover the rel-

evant theory on Godunov schemes, non-ideal thermodynamics, and machine learning. §4 analyzes preliminary designs of different FluxNet approaches, and comments on the feasibility that these FluxNets could offer competitive performance (*i.e.* balance of accuracy and efficiency) relative to traditional Riemann solvers. In §5, the data preparation process is discussed, including the generation and post-processing of exact data used for training and testing. A total of six FluxNet designs detailed in §6 are trained and tested. Their learning curves are studied in §7.1, with the final two candidates tested on a transcritical nitrogen shock tube in §7.2 and the final physics-informed FluxNet tested on a two-dimensional Riemann problem in §7.3. Final commentary on the accuracy and runtime efficiency of exact, linearized, and FluxNet approaches is discussed in §7.4. This work comprises Chapter 6 of the doctoral thesis by Wang [28, 29].

2. Computational Fluid Dynamics

2.1. Governing Equations

This study considers unsteady 1D and 2D flows. The flow is assumed to be inviscid and isentropic. Therefore, the time-dependent Euler equations apply:

$$\mathbf{U}_t + \mathbf{F}(\mathbf{U})_x + \mathbf{G}(\mathbf{U})_y = 0 \quad (1)$$

where $\mathbf{F}$ and $\mathbf{G}$ are the fluxes in the x and y directions, which depend on the conserved variables $\mathbf{U}$. Also, t is time. The flux and conservative vectors may be expanded as:

$$\mathbf{U} = \begin{bmatrix} \rho \\ \rho u \\ \rho w \\ E \end{bmatrix}; \quad \mathbf{F} = \begin{bmatrix} \rho u \\ p + \rho u^2 \\ \rho u w \\ u(E + p) \end{bmatrix}; \quad \mathbf{G} = \begin{bmatrix} \rho w \\ \rho w u \\ p + \rho w^2 \\ w(E + p) \end{bmatrix} \quad (2)$$

where ρ is density, u is the velocity in the x -direction, w is the velocity in the y direction, p is pressure, and the total energy per unit volume E is:

$$E = \frac{e}{v} + \frac{1}{2}\rho(u^2 + w^2) \quad (3)$$

where e is specific internal energy on a molar basis, $v = M/\rho$ is the molar specific volume, and M is the molar mass of the fluid. It is also possible to express the specific enthalpy on a molar basis as $h = e + pv$. The total

enthalpy H on a per unit mass basis may then be calculated as $H = (E+p)/\rho$. For 1D flow, the $\mathbf{G}$ and w terms are zero.

2.2. Thermodynamics

Transcritical flows are characterized by conditions of high pressure where it is necessary to account for intermolecular forces in the thermodynamics. In such cases, the ideal gas assumptions do not hold and a more general equation of state is required. In this study, the Peng-Robinson [30] cubic state equation is used as it permits direct comparison with analytical and approximate solutions to the 1D Riemann problem [31]:

$$p = \frac{RT}{v-b} - \frac{\Theta(T)}{v^2 + 2bv - b^2} \quad (4)$$

where R is the universal gas constant, T is temperature, Θ is a function of temperature, and b is a constant related to the fluid. For nitrogen gas as considered in this study, the cubic parameters were calculated specifically as:

$$a = 0.45724 \frac{R^2 T_c^2}{p_c} \quad (5)$$

$$b = 0.07780 \frac{RT_c}{p_c} \quad (6)$$

$$\Theta(T) = a \left(1 + (0.37464 + 1.54226\omega - 0.26990\omega^2) \left[1 - \left(\frac{T}{T_c} \right)^{0.5} \right] \right)^2 \quad (7)$$

in which $T_c = 126.2\text{K}$, $p_c = 3.398\text{MPa}$, $\omega = 0.0370$ [32, 33]. This work is restricted to single-species fluids, however Poling *et al.* [32] offer guidance on modelling mixtures. Pressure-explicit state equations, such as Eq. (4), are desirable because the Maxwell relations enable all thermodynamic quantities to be expressed in terms of explicit partial derivatives and integrals of pressure.

With a particular state equation selected, the specific heat capacities, enthalpy, and speed of sound may be expressed as functions of v, T by applying thermodynamically consistent departure functions to the corresponding ideal gas states [32]. The resulting equations for non-ideal specific heat capacities, enthalpy, and speed of sound are analytical except for a polynomial fit of ideal-gas constant-pressure specific heat capacity, for which polynomials of

degree 6 have been shown to yield errors within 0.5% of experimental values [34]. Detailed derivations are available in earlier studies [35, 31].

2.3. Numerical Method

When studying Riemann solvers, it is helpful to utilize a first-order discretization scheme. This is because simple schemes most clearly isolate the effects of the Riemann solver itself while permitting errors, non-physical features, and instabilities to be studied [31]. Nevertheless, the findings may be trivially extended to higher-order schemes which, too, leverage Riemann solvers. By applying a cell-centred unsplit finite volume analysis with a uniform Cartesian grid [22] to Eq. (1), it is possible to arrive at the following first-order upwind Godunov scheme:

$$\mathbf{U}_{i,j}^{n+1} = \mathbf{U}_{i,j}^n + \frac{\Delta t}{\Delta x} \left(\mathbf{F}_{i-\frac{1}{2},j} - \mathbf{F}_{i+\frac{1}{2},j} \right) + \frac{\Delta t}{\Delta y} \left(\mathbf{G}_{i,j-\frac{1}{2}} - \mathbf{G}_{i,j+\frac{1}{2}} \right) \quad (8)$$

where the superscripts indicate the index of the time step and the subscripts indicate the spatial cell under consideration. The Riemann problem is thus a foundational component to numerical methods in compressible flows, as it can be used to compute the intercell flux terms at each time step [36].

In the Riemann problem, two constant flow states are initially separated by an imaginary membrane at $x = 0$. At $t = 0$, the imaginary membrane disappears and the solution at any x and $t > 0$ may be found by solving the one-dimensional, time-dependent Euler equations for the given initial conditions. The self-similar structure of solutions to the Riemann problem has been studied extensively in literature [37, 36, 38, 39, 35], and this characteristic wave pattern is frequently leveraged to find the intercell flux at $x = 0$.

Traditionally, solutions to the Riemann problem are computed with either an exact or approximate (*e.g.* linearized) solver. Exact solvers require multiple iterations to achieve a high degree of accuracy [40, 39, 41], especially when non-ideal thermodynamics are of interest. This is useful for generating reference data to test numerical codes. However, practical CFD simulations may involve many thousands or millions of calls to a Riemann solver. Hence, approximate solvers must carefully tradeoff accuracy versus computational simplicity. Typical approximate solvers are based on a linearized form of the governing equations [42, 43, 44]. Machine learning presents a third possible category of Riemann solver that is explored in this paper, which strives to

balance the accuracy and computational demands of the usual approaches.

In this study, the above-noted first-order Godunov scheme is modified to include strong stability-preserving third-order Runge-Kutta time integration [45]. FluxNets are developed and trained to predict variables used in calculating the fluxes, and the results are compared to fluxes computed with the well-known Roe solver [42] and Roe-StARS solver for entropy-stable simulations [31]. Exact solutions to the transcritical shock tube test case of §7.2 are computed with a Collela & Glaz-type solver [40].

3. Machine Learning

3.1. Supervised Learning

Mitchell’s [46] definition of ML may be stated as the phenomenon by which a computer program improves its performance P on one or more tasks T from experience E . The program is said to *learn* from experience E , which contains *training data*. When the trained program is used to make predictions for practical purposes, this process is referred to as *inference*. Numerous algorithms have been developed over the years to realize learning behaviour in computer programs, and they may be generally classified as supervised, unsupervised, and semi-supervised learning [47]. For a Riemann solver, the task T is to map initial conditions to intercell fluxes by learning from training data E produced by exact-iterative computations—thus, a supervised learning approach is appropriate.

In the context of Riemann solvers, the input or feature vector $\mathbf{x}_i$ should contain sufficient information from the initial conditions in order to uniquely determine the flux. This paper employs a feature vector of the form $\mathbf{x}_i = [\rho_L, u_L, p_L, \rho_R, u_R, p_R] \in \mathbb{R}^6$, and the prediction or output vector is chosen to be $\mathbf{y}_i = [\rho_{*L}, u_{*L}, p_{*L}, u_*, \rho_{*R}, u_{*R}, p_{*R}] \in \mathbb{R}^7$. The subscripts L, R refer to the initial left and right conditions, while $*L, *R$ refer to the left and right star-state conditions. At first glance, it may seem odd to include u_*, u_{*L}, u_{*R} when all three are constant in the star-state—however this separation is necessary to enforce physical constraints as discussed later in §3.4. From these predicted variables, it is possible to determine the wavespeeds in the Riemann problem and estimate the intercell flux. §4 contains a comprehensive analysis on the possible choices of feature and predicted variables, and how they may be integrated with the broader numerical scheme.

The supervised learning task at hand can thus be made more specific: to find a regression model $\mathcal{F}(\mathbf{x}_i; \theta) \in \mathbb{R}^{6 \times 7}$ with learnable parameters or weights

θ that approximates the true mapping from initial conditions to star-state conditions as stated above. The regression model's L_1 error, which ultimately impacts the accuracy of the wavespeed and flux estimates in the numerical scheme, is:

$$\Delta y_i = \sum_j |\hat{y}_{ij} - y_{ij}| \quad (9)$$

where $\hat{\mathbf{y}}_i = \mathcal{F}(\mathbf{x}_i; \theta)$ is the neural network prediction. Only nitrogen gas is studied, although any single-species gas may be used. For mixtures, additional parameters would need to be represented in the feature vector.

3.2. Multi-Layer Perceptron

A feed-forward neural network architecture with non-linear activation functions and fully connected hidden layers is used. This architecture, known as the multi-layer perceptron, was selected due to its popularity when performing non-linear regression tasks in fluid mechanics [48, 25, 23, 49, 50]. More complex networks such as convolutional and recurrent networks are found in image processing, speech analysis, and natural language processing [18] where the focus is instead on classification and ordered data. However for low-dimensional continuously real-valued inputs and outputs, the multi-layer perceptron is simple yet suitable.

The architecture of a generic multi-layer perceptron for the Riemann problem is shown in Fig. 1. There are $k + 1$ layers (*i.e.* a depth of $k + 1$) of which k are hidden layers, while each layer has a width of j which may vary from layer to layer. For simplicity, we consider network designs where j is constant across all layers, similar to Magiera *et al.* [23] and Gyrya *et al.* [25]. Each linear transform is of the form $\mathbf{z} = \mathbf{A}\mathbf{h} + \mathbf{b}$ where $\mathbf{A} \in \mathbb{R}^{j \times j}$ and $\mathbf{z}, \mathbf{h}, \mathbf{b} \in \mathbb{R}^j$, which is equivalent to the more convenient form:

$$\mathbf{z}_k = \tilde{\mathbf{A}}_k \tilde{\mathbf{h}}_k \quad (10)$$

where $\tilde{\mathbf{A}} = [\mathbf{A} \ \mathbf{b}] \in \mathbb{R}^{j \times j+1}$ and $\tilde{\mathbf{h}} = [\mathbf{h} \ 1]^T \in \mathbb{R}^{j+1}$. For the first linear layer, $\tilde{\mathbf{h}}_1 = [\mathbf{x}_1 \ 1]$. For the $(k + 1)$ th layer, $\mathbf{z}_{k+1} = \hat{\mathbf{y}}_i$. Within the k^{th} hidden layer, an element-wise non-linear activation function ϕ_k is applied:

$$\tilde{h}_{kj} = \phi_k(z_{kj}) \quad (11)$$

Common activation functions are sigmoid, tanh, radial basis, and the rectified linear unit [51, 52]. To avoid potential saturation issues with vanishing

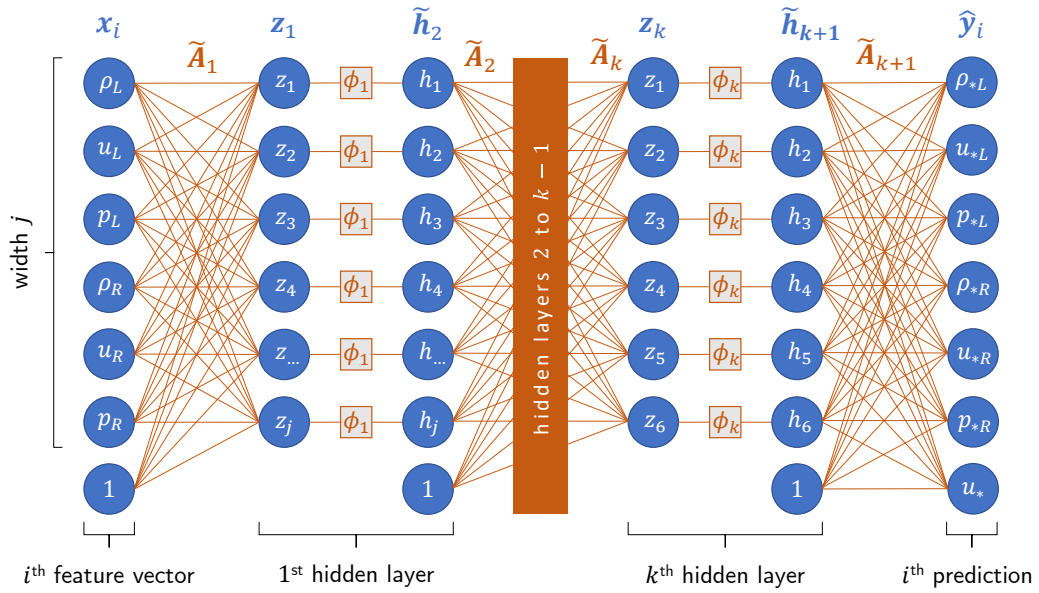


Figure 1: A generic multi-layer perceptron designed for the Riemann problem with depth $k + 1$ and width j at each layer (which need not be equal to the dimension of $\mathbf{x}_i$ nor $\hat{\mathbf{y}}_i$). Each hidden layer applies a non-linear activation function in an element-wise fashion. The matrices $\tilde{\mathbf{A}}$ contain the weights of the network. The number of weights to be trained scales as $O(jjk)$.

gradients, a leaky rectified linear unit (Leaky ReLU) activation function with slope 10^{-2} is used.

It is also worth noting that the training dataset $(\mathbf{x}_i, \mathbf{y}_i)$ typically needs to be scaled in order to prevent saturation of activation functions, particularly when there are different orders of magnitude or units in the feature and prediction vectors. For this study, mean normalization is used. Alternative normalization schemes include Z-score and min-max normalization, however we know *a priori* that the data is not normally distributed, while min-max normalization rescales inputs to $[0, 1]$ which does not take advantage of Leaky ReLU’s nonzero outputs for negative inputs.

3.3. Loss Function and Optimizer

In addition to specifying the network design, it is necessary to determine an appropriate loss function and optimizer. The loss function is designed to take on smaller values when the weights θ yield better performance, *i.e.* lower values of some error metric of interest. Training involves executing the search algorithm, also called the optimizer, to find weights that minimize the loss function for some set of training data. For every data point $(\mathbf{x}_i, \mathbf{y}_i)$ and choice of weights θ , there is a loss $\mathcal{L}$ given by some loss function $\mathcal{L}(\mathbf{x}_i, \mathbf{y}_i; \theta)$ that represents a penalty related to poor performance. Often, the loss is taken as a mean average across the training data:

$$\mathcal{L}(\mathcal{D}; \theta) = \frac{1}{n} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}} \mathcal{L}(\mathbf{x}, \mathbf{y}; \theta) \quad (12)$$

where $\mathcal{D} = \{(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_n, \mathbf{y}_n)\}$ is the dataset used for training. Common loss functions for regression include the L_2 norm, Huber, as well as Mean-Squared Error (MSE) [51, 52]. Due to the suitability of MSE for generic regression problems, it is utilized in this study:

$$\mathcal{L}_{MSE}(\mathcal{D}; \theta) = \frac{1}{n} \sum_{(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}} \|\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \theta)\|^2 \quad (13)$$

It is worth emphasizing that whereas the present loss functions are used for training, the L_1 error from Eq. (9) is a performance metric that is used only to evaluate the error that propagates into the numerical scheme based on the star-state predictions. An optimization algorithm may then be applied that searches for weights θ such that the loss function is minimized

over the dataset $\mathcal{D}$. Common optimizers include stochastic gradient descent, root-mean square propagation [53], and Adam [54], among others. This study employs the Adam [54] optimizer, named after the algorithm’s use of **adaptive moments**. By varying the learning rate via first and second gradient moments with exponential decay, Adam facilitates fast convergence even in high-dimensional non-linear search spaces such as those expected in the Riemann problem with non-ideal thermodynamics.

3.4. Regularization and a Physics-Informed Approach

The universal approximation property [55, 56, 57] guarantees the existence of $\mathcal{F}$, however finding weights θ that yield a desired level of accuracy is generally an NP-hard problem. Thus, the tuning of hyperparameters (*i.e.* parameters that control the learning process), selection of appropriate train and test data, and use of regularization techniques (*e.g.* ridge regression, lasso, data augmentation, bagging, dropout, batch normalization) are additional considerations for mitigating overfitting and reducing generalization error. A balance must be struck between accuracy on the training data versus accuracy on unseen data that may be encountered during inference.

Since the present research involves a regression task governed by known physical laws, traditional data-driven regularization techniques are eschewed in favour of a *physics-informed* approach. A physics-informed approach [20, 19] involves directly integrating physical laws into the neural network or loss function such that physical violations are penalized during training. Unlike data-driven regularization, physical constraints are free of subjective heuristical criteria for smoothness and generality of predictions. Weights that yield low losses on training data are still penalized if they violate physical laws, just as how traditional regularization penalizes weights that might yield higher losses on unseen test data. As with all regularization techniques, the physics-informed approach enables smaller training datasets to be used for accurate predictions across a wide feature space.

Physical constraints are incorporated through a CAL approach [23]. Alternatively, the network complexity itself may be increased, but this would reduce runtime efficiency. A generic CAL loss function is of the form:

$$\mathcal{L}_{CAL}(\mathcal{D}; \theta) = \frac{1}{n} \sum_{(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}} ||\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \theta)||^2 + \kappa |\psi(\mathbf{x}_i; \theta)| \quad (14)$$

where κ is a user-defined hyperparameter and $\psi(\mathbf{x}_i; \theta) \in \mathbb{R}$ is the residual

calculated from a physics-informed constraint function such that $\psi = 0$ when the constraint is satisfied.

A natural set of physical constraints for the Riemann problem is furnished via the Rankine-Hugoniot jump conditions applied to the Euler equations (Eq. 1). Standard textbooks such as [22] derive the jump conditions as:

$$u_s (\mathbf{U}(x_1, t) - \mathbf{U}(x_2, t)) + \mathbf{F}(\mathbf{U}(x_2, t)) - \mathbf{F}(\mathbf{U}(x_1, t)) = 0 \quad (15)$$

where $u_s = dx_s/dt$ is the speed at which the jump is propagating. Eq. (15) may be reformulated:

$$u_s = \frac{\mathbf{F}(\mathbf{U}(x_2, t)) - \mathbf{F}(\mathbf{U}(x_1, t))}{\mathbf{U}(x_2, t) - \mathbf{U}(x_1, t)} \quad (16)$$

and it is easily observed that the faster that the flux changes with respect to the conserved variables, the faster the jump speed—as one might intuitively expect.

The Rankine-Hugoniot constraints are conveniently prescribed at the contact discontinuity in the Riemann problem, since velocity and pressure remain constant across such waves. In terms of the variables that $\mathcal{F}$ is designed and trained to predict, it may be formally stated that $p_{*L} = p_{*R}$ and $u_{*L} = u_{*R} = u_*$. Expanding and rearranging Eq. (16) yields individual constraint functions for mass, momentum, and energy:

$$\psi_{mass} = u_s(\rho_{*R} - \rho_{*L}) - \rho_{*R}u_{*R} + \rho_{*L}u_{*L} \quad (17)$$

$$\psi_{mom} = u_s(\rho_{*R}u_{*R} - \rho_{*L}u_{*L}) - (\rho_{*R}u_{*R}^2 + p_{*R}) + (\rho_{*L}u_{*L}^2 + p_{*L}) \quad (18)$$

$$\psi_{energy} = u_s(E_{*R} - E_{*L}) - u_{*R}(E_{*R} + p_{*R}) + u_{*L}(E_{*L} + p_{*L}) \quad (19)$$

and it can be verified through direct substitution that $\psi_{mass}, \psi_{mom}, \psi_{energy} = 0$ across a contact discontinuity. Should $\mathcal{F}(\mathbf{x}_i; \theta)$ incorrectly predict one or more star-state conditions of interest, a nonzero physics-informed penalty is applied as follows:

$$\psi(\mathbf{x}_i; \theta) = \kappa_{mass}\psi_{mass}(\mathbf{x}_i; \theta) + \kappa_{mom}\psi_{mom}(\mathbf{x}_i; \theta) + \kappa_{energy}\psi_{energy}(\mathbf{x}_i; \theta) \quad (20)$$

where the $\kappa_{mass}, \kappa_{mom}, \kappa_{energy}$ terms are additional user-defined hyperparameters that weigh the relative importance of each constraint in the training process.

Furthermore, the total energy on a per unit volume basis E is neither a feature nor prediction of $\mathcal{F}$, so it is necessary to restore the original magnitudes of the variables to calculate the total energy terms. The resultant values can subsequently be normalized by some factor suitable for the training data in use. Also, the energy constraint is the most functionally complex of the three constraints, since energy calculations require the evaluation of the energy-enthalpy relationship and non-ideal enthalpy, which in turn depend on several thermodynamic departure functions. This issue does not arise in existing literature, which is based upon the perfect gas assumption. But for non-ideal thermodynamics, preliminary training attempts in §4 revealed that omitting the energy constraint improved convergence and accuracy. The energy constraint is therefore not used in this study.

4. Preliminary Design and Feasibility Assessment

Prior to the systematic training and evaluation of any FluxNets, it is wise to thoroughly consider the alternative choices for feature and prediction vectors, their potential implications for network performance, and how the resulting network must be integrated with the broader numerical scheme. As part of this analysis, preliminary, informal training attempts were conducted in Python 3.9 and PyTorch 1.10.2, using the datasets described shortly in §5. These initial training sessions helped to determine which candidate designs were suitable for continued study. The dimensions used for preliminary training were 6 hidden layers with 64 nodes per hidden layer, benchmarked on the multi-layer perceptron designs in existing literature [25, 23].

It is crucial to note that if calculations are parallelized, then the time complexity of multiplying two square matrices is $O(\ln(n))$ where n is the height or width of each matrix [58]. Therefore, a multi-layer perceptron with n nodes per layer and h hidden layers would have a time complexity τ that scales as:

$$\tau = h(\ln(n) + n) \quad (21)$$

in which the addition of n accounts for the element-wise activation functions per layer. So, a 64×6 FluxNet scales as $O(6(\ln(64) + 64)) \approx 409$. A further $O(10^2)$ operations are required to compute the intercell flux from the star-state conditions predicted by each FluxNet. Thus, a 64×6 FluxNet possesses a time complexity of roughly 509, as compared to the 331 to 387 floating point operations when computing the Roe-StARS flux in the case

of a Peng-Robinson gas [31]. In traditional Riemann solvers, many of these calculations are nonlinear and therefore must be performed serially rather than in matrix form. Although an ML approach does indeed require more total computations, the runtime consequences can be mitigated through parallelized linear algebra realized on GPU-equipped servers. A mild sensitivity analysis of network performance to network size is discussed later in the systematic training and evaluation phase beginning in §6. If the 64×6 initial network specification produced inadequate performance during the preliminary design phase, then hyperparameters and dimensions were varied until the network complexity exceeded or would exceed that of traditional solvers by an order of magnitude. In this case, the design was abandoned. The number of nodes in each layer was held fixed in this study in order to limit the search space, however a sensitivity analysis could be conducted in the future.

A total of four candidates for feature and prediction vectors were examined (Tab. 1). Although finer variations are possible, these four represent distinct philosophies in the development of a learning-based Riemann solver: 1) a conservative variable-only approach, 2) a primitive-to-conservative solver, 3) a purely primitive variable solver that specifies all independent thermodynamic state variables, and 4) a primitive variable solver with only the minimum required predictions. The analysis herein concludes that it is indeed feasible to develop a learning-based Riemann solver whose accuracy and computational complexity fall in between current approximate and exact approaches, with the fourth design eventually selected for the development of detailed FluxNet candidate designs in §6. The first three candidates were discarded for theoretical reasons and initial training observations that will be discussed below.

Table 1: Potential choices of feature and prediction vectors for designing a FluxNet (*i.e.* a learning-based Riemann solver). The subscripts (L, R) refer to the left and right initial conditions, while $(*L, *R)$ refer to the left and right star-state conditions. $\mathbf{F}$ with no subscript is shorthand for the intercell flux, that is, $\mathbf{F}(\mathbf{U}_{i+\frac{1}{2}})$ assuming that (L, R) correspond to the $(i, i + 1)$ cells.

Design No.	Features (Input)	Predictions (Output)
1	$\mathbf{U}_L, \mathbf{U}_R$	$\mathbf{F}, u_*$
2	$p_L, u_L, \rho_L, p_R, u_R, \rho_R$	$\mathbf{F}, u_*$
3	$p_L, u_L, \rho_L, T_L, p_R, u_R, \rho_R, T_R$	$\rho_{*L}, u_{*L}, p_{*L}, T_{*L}, u_*, \rho_{*R}, u_{*R}, p_{*R}, T_{*R}$
4	$p_L, u_L, \rho_L, p_R, u_R, \rho_R$	$\rho_{*L}, u_{*L}, p_{*L}, u_*, \rho_{*R}, u_{*R}, p_{*R}$

Design 1 represents an end-to-end Riemann solver in the sense that no further mathematical steps are required to compute each step of the Godunov scheme as shown in Eq. (8). In Design 1, it is possible to go directly from the known conservative variables at one time step, to the desired conservative variables at the next time step, using a direct mapping from $\mathbf{U}$ to $\mathbf{F}$. This design lends itself to a rather simple numerical scheme by using entirely conservative variables and eliminating the usual computational overhead of alternating between conservative variables (useful for fluid dynamics) and primitive variables (useful for thermodynamics), which become highly computationally expensive under non-ideal thermodynamic conditions. The Rankine-Hugoniot conditions, in conservative form (Eq. 20), may be used to define a physics-informed loss function.

Despite its seeming simplicity and elegance, Design 1 possesses two critical flaws. Firstly, recall that the Courant-Friedrichs-Lewy (CFL) condition requires the speed of sound to be computed at each time step. However, the speed of sound is a thermodynamic quantity that, in general, depends on the complete thermodynamic state defined by two independent primitive variables, *e.g.* v, T when using cubic state equations. Although Design 1 would simplify flux computations, the overall numerical scheme still requires the overhead of conservative-to-primitive conversion. Secondly, the use of conservative variables forces the FluxNet to discover the mapping from conservatives to primitives and vice-versa. This is because the characteristic waves that influence the behaviour of the Riemann problem are functions of primitive variables, and there are multiple combinations of primitive variables that correspond to the same momentum and energy components of $\mathbf{U}$. Design 1 requires that the FluxNet learn the complete primitive to conservative mapping in order to preserve a one-to-one correspondence. Design 1 would therefore increase network complexity while simultaneously still requiring primitives to be calculated separately for CFL purposes.

Design 2 attempts to overcome the weaknesses of Design 1 by mapping primitive variables to the intercell flux $\mathbf{F}$, eliminating the need for the FluxNet to uncover the highly nonlinear conservative-to-primitive relationship. In practice, attempts to train a multi-layer perceptron to predict $\mathbf{F}$ were unfruitful. Despite varying hyperparameters and testing other optimizers, it was observed that network widths and depths more than triple the initial dimensions could not reduce training or test errors below 30%. An example of a preliminary training run on Design 2 is shown in Fig. 2—summing the left and right flux components gives the resulting component of the intercell

flux of interest $\mathbf{F}$. This poor performance may be attributed to the sheer complexity of the underlying mathematical relationships. The flux terms are nonlinear expressions of primitive variables that depend on which state of the Riemann problem encloses the cell interface. The correct state is, in turn, dependent on the accurate estimation of characteristic waves that separate the states, which further depend on the complex mathematical form of the non-ideal speed of sound. Indeed, if a FluxNet could be designed to predict fluxes accurately, it is likely that inference would be far slower than running exact-iterative solvers, let alone approximate solvers, even when computations are parallelized. Fuks & Tchelepi [26] have also noted difficulties in predicting flux variables from initial conditions, however that was in the context of two-phase porous media.

In Design 3, a primitive-to-primitive mapping is proposed that also includes T , which together with ρ , may be used to calculate the wavespeeds of the Riemann problem, determine which region of the Riemann problem occupies the intercell boundary, and compute the flux. It should be noted that estimating all three thermodynamic state variables does not necessarily over-constrain the prediction space, provided that the network learns the underlying state equation. The Rankine-Hugoniot constraints in primitive form (Eq. 15) may be employed to define a physics-informed loss function. Once again, practical attempts to train such a network revealed difficulties in predicting p, ρ, T that were consistent with the state equation. Despite varying hyperparameters, network width and depth, and optimizers, the best-performing FluxNets predicted only two of three thermodynamic state variables within $< 30\%$ error during any training session. The percent error in the third remaining state variable was observed oscillating near $O(10^2)$. An example of a preliminary training run on Design 3 is shown in Fig. 2. An additional loss term based on the state equation was tested, thereby incorporating an explicit constraint to assist learning of the state equation, but yielded similar results. As with Design 2, Design 3 experienced difficulty producing physically consistent predictions in an efficient manner.

Design 4 was selected for further exploration. By removing T , initial training attempts with a 64×6 FluxNet revealed that all predicted star-state variables fell well below 5% error after a mere 200 epochs, with potential for improvement as the loss function was still decreasing at the point where the training process was stopped prematurely for the purposes of preliminary evaluation. It is worth acknowledging that a version of Design 4 was trained to predict ρ, T rather than p, ρ , but like with Design 3, either ρ or T

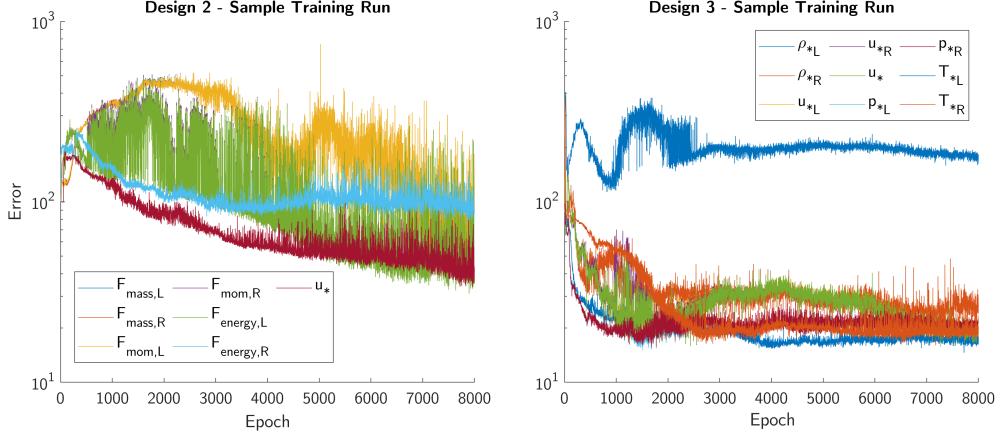


Figure 2: Learning curves from sample training runs on Design 2 (left) and Design 3 (right), showing mean L_1 test errors. In Design 2, the errors show significantly oscillation and do not fall below 30%, while in Design 3, the errors stall around 12% in all predicted variables except T_{*L} which oscillates at $> 100\%$ error. Accordingly, neither Design 2 nor Design 3 were advanced further.

tended to diverge. Compared to Design 3, the only difference in the broader numerical scheme is that Design 4 requires a post-inference computation of T_{*L} and T_{*R} from the corresponding p, ρ using the state equation. It is also interesting that although the physics of the Riemann problem depends on the choice of state equation, the absence of T and any state equation-based physical constraint produced improved convergence, stability, and accuracy. Moreover, this performance was achieved only with the mass and momentum constraints. In fact, the inclusion of the energy constraint hampered the convergence and stability of the training process due to the extremely nonlinear relationships underpinning the energy terms.

It should be emphasized that the accurate prediction of star-state conditions and wavespeeds in turn leads to more accurate numerical results. This is true not only of transcritical and transonic conditions, but compressible flows in general. For instance, the HLLC solver [44] improved resolution of material interfaces and sharp flow features by restoring the contact discontinuity wave to the star-state estimates. Hence, the remaining ML research focuses primarily on the network development process as it relates to accurate and efficient prediction of the star state, culminating in a perfunctory demonstration of FluxNets for solving the transcritical shock tube problem investigated in Ma *et al.* [59] and subsequently Wang & Hickey [31].

5. Data Preparation

Datasets for training and testing were generated using a MATLAB code that iteratively computes exact solutions to the non-ideal Riemann problem using a Collela & Glaz [40] scheme modified to use the analytical derivations of Wang & Hickey [35]. Each sample involved 2 to 6 iterations to solve the Riemann problem within $< 0.1\%$ error in all primitive star-state variables. Intercell fluxes were also computed for the preliminary training attempts on Design 2 of §4. Many samples failed to converge since the Newton-Raphson iterations therein guarantee neither stability nor convergence, especially in the case of highly nonlinear non-ideal thermodynamics.

The thermodynamic state space around the critical point for nitrogen gas was sampled a total of 64,000 times using uniform probability distributions across the 6 feature variables representing initial conditions. Velocities were selected so that sub-, trans-, and supersonic conditions would be represented. The limits of sampling are shown in Tab. 2. It is important to note that the range of training data in this study was selected to encapsulate the highly nonlinear behaviour of thermodynamic variables about the pseudo-boiling curve [60]. Network performance, and thus network size and complexity, will generally vary depending on the desired range of features and predictions that the network is expected to map accurately.

Of the 64,000 points for which exact solutions were attempted, only 21,237 points or approximately one-third converged. Liquid points were then removed (i.e. points above the boiling curve but below the critical temperature). Repeated points were also deleted to ensure a uniform distribution of samples. The resulting dataset comprised $2^{13} = 8192$ points, which was doubled to 16,384 points by swapping left and right initial conditions via symmetry of the Riemann problem. Finally, the data was mean-normalized to avoid gradient saturation during the eventual training process. Training and test splits were taken with ten-fold cross-validation on a randomly sampled subset of 16,000 training and 1,600 test points. Although this data preparation approach was used for nitrogen gas assuming Peng-Robinson state equation, it is expected to hold for other gases since the corresponding states principle involves shifting the critical locus but maintaining the mathematical form of the thermodynamic relationships [32].

Table 2: Ranges of initial conditions represented in the training and test datasets. The ranges are representative of transcritical flow problems [59].

Feature	Range
p_L, p_R	100 kPa to 13.5 MPa
u_L, u_R	-200 to 200 ms ⁻¹
ρ_L, ρ_R	1 to 200 kg m ⁻³

6. Network Architectures and Losses

Six variations in multi-layer perceptron architecture and loss function were trained and evaluated (Tab. 3), all based on Design 4 from §4. These specific architectures and losses were chosen to assess the impact of network complexity and loss function on performance. Network depth was varied in order to evaluate the sensitivity of final losses and errors to the network complexity—a more exhaustive investigation of network size is left to future studies, as the current research is framed around feasibility and practicality, not optimality. Network size also varies depending on the range of initial conditions sampled and the nonlinearities therein. Each FluxNet was trained via both an MSE loss (Eq. 13) and a CAL version of MSE (Eq. 14), in order to compare traditional and physics-informed approaches. The CAL version included a Rankine-Hugoniot physical constraint of the form Eq. (20) with mean-normalized variables, $\kappa_{mass}, \kappa_{mom} = 1$, and $\kappa_{energy} = 0$ thus omitting the energy constraint. For non-ideal thermodynamics, the energy constraint was determined to have a deleterious effect on training as stated in §3.4 and §4.

All FluxNets were trained in double-precision Python 3.9 and PyTorch 1.10.2 using an Adam optimizer. The optimizer was configured with learning rate 2.5×10^{-5} , first and second moment decay rates 0.9 and 0.999, and weight decay rate 10^{-9} . For the physics-informed loss functions, $\kappa = 10^{-6}$ was set in Eq. (14). These hyperparameters were empirically found to yield adequate stability and convergence based on log-uniform sampling followed by a sensitivity analysis of hyperparameters at 5% intervals within each order of magnitude. A maximum of 8000 training epochs were conducted per FluxNet, with the final weights selected based on the lowest training loss obtained in the last 4000 epochs. In order to control for varying weight initializations and choices of cross-validation splits, the same random seed was used for all FluxNets. However, the training process was repeated a

second time *a posteriori* with a different seed to verify that results were independent of seeding. All data preparation, network training, and inference were performed on an MSI GF Series GF65 144Hz Intel i7-10750H laptop equipped with NVIDIA GeForce RTX 3060, 16 GB memory, 512 GB PCIe SSD, and 64-bit Windows 10.

Table 3: List of FluxNet architectures and loss functions evaluated in this study. The time complexity refers to the number of temporally distinct floating point operations assuming parallel inference, including an additional $O(10^2)$ operations required to compute the intercell flux from the star-state conditions predicted by each FluxNet. This overhead can vary slightly depending on the number of $T = T(p, \rho)$ iterations required.

Name	Nodes	Hidden Layers	Loss	Time Complexity
64×5 MSE	64	5	MSE	441
64×5 MSE-RH	64	5	MSE + ϕ	441
64×6 MSE	64	6	MSE	509
64×6 MSE-RH	64	6	MSE + ϕ	509
64×7 MSE	64	7	MSE	577
64×7 MSE-RH	64	7	MSE + ϕ	577

7. Results

Learning curves and numerical results are presented for the learning-based Riemann solvers considered above. The learning curves show the progression of errors and losses during the training process of each neural network, revealing that the training process for all networks seem similarly promising in terms of mean L_1 errors. The FluxNet approach also yields a significantly tighter distribution of L_1 errors as compared to the Roe family of solvers. Moreover, the numerical results indicate the importance of a physics-informed approach while demonstrating improved numerical stability and accuracy relative to Roe-type solvers. The MSE and MSE-RH versions of the 64×5 FluxNet were used to solve a transcritical shock tube test case, showing that the 64×5 MSE-RH FluxNet yielded smoother, more generalizable solutions. A two-dimensional Riemann problem was also solved in which the Roe solution admits various nonphysicalities and errors that were mitigated by the 64×5 MSE-RH FluxNet. Overall, the 64×5 MSE-RH neural network achieved an order of magnitude improvement in accuracy with a 23% increase in time complexity relative to Roe-StARS, assuming parallelized computations.

7.1. Learning curves

The learning curves for each FluxNet in Tab. 3 are shown in Figs. 3 and 4. Plotted are the progression of test errors, train losses, and test losses across all epochs. The vertical axes are logarithmic to help easily distinguish the curves from each other—careful interpretation is therefore warranted since the seemingly large spikes are in fact relatively small in magnitude. The final errors and losses are compiled in Tab. 4. Mean relative L_1 errors across the test dataset are reported. The mean relative L_1 errors are also tabulated for the Roe solver and the Collela & Glaz [40] iterative solver, using the same dataset on which the FluxNets were tested.

Table 4: Mean relative L_1 test errors (% of the exact solution) and losses (unitless) for six different learning-based Riemann solvers. For reference, typical errors for approximate and exact-iterative solvers are shown.

Solver	ρ_{*L}	u_{*L}	p_{*L}	u_*	ρ_{*R}	u_{*R}	p_{*R}	$\mathcal{L}_{train}[10^{-6}]$	$\mathcal{L}_{test}[10^{-6}]$
64×5 MSE	0.28	0.09	0.11	0.10	0.17	0.10	0.09	3.14	1.44
64×5 MSE-RH	0.29	0.08	0.17	0.07	0.22	0.08	0.13	3.21	1.40
64×6 MSE	0.26	0.09	0.15	0.12	0.19	0.09	0.16	3.04	1.45
64×6 MSE-RH	0.25	0.13	0.17	0.14	0.22	0.14	0.17	2.94	1.43
64×7 MSE	0.28	0.14	0.13	0.13	0.27	0.13	0.16	2.92	1.51
64×7 MSE-RH	0.94	1.08	0.96	1.05	0.84	1.13	0.77	5.82	3.7
Roe	4.20	1.04	1.74	1.04	0.27	1.04	1.74	N/A	N/A
Exact	0.01	0.01	0.01	0.01	0.01	0.01	0.01	N/A	N/A

All FluxNets except 64×7 MSE-RH achieved final test errors of $O(10^{-1})\%$. The networks tended to elicit greater errors in density (around 0.2 to 0.3%) and smaller errors in velocity and pressure (around 0.1 to 0.2%). The lower errors on velocity and pressure are likely due to physics of the contact discontinuity wave, across which pressure and velocity should be constant. Such lower errors are observed for both the MSE and MSE-RH FluxNets, which suggests that the networks were able to learn this pattern at least partially from the data alone. By contrast, density can change across a contact discontinuity (*i.e.* since the contact discontinuity represents the moving material interface between the initial left and right states). Learning of the pressure-density relationship may have been hampered by the absence of the Rankine-Hugoniot energy constraint—which, again, was omitted due the highly nonlinear physics causing unstable training behaviour.

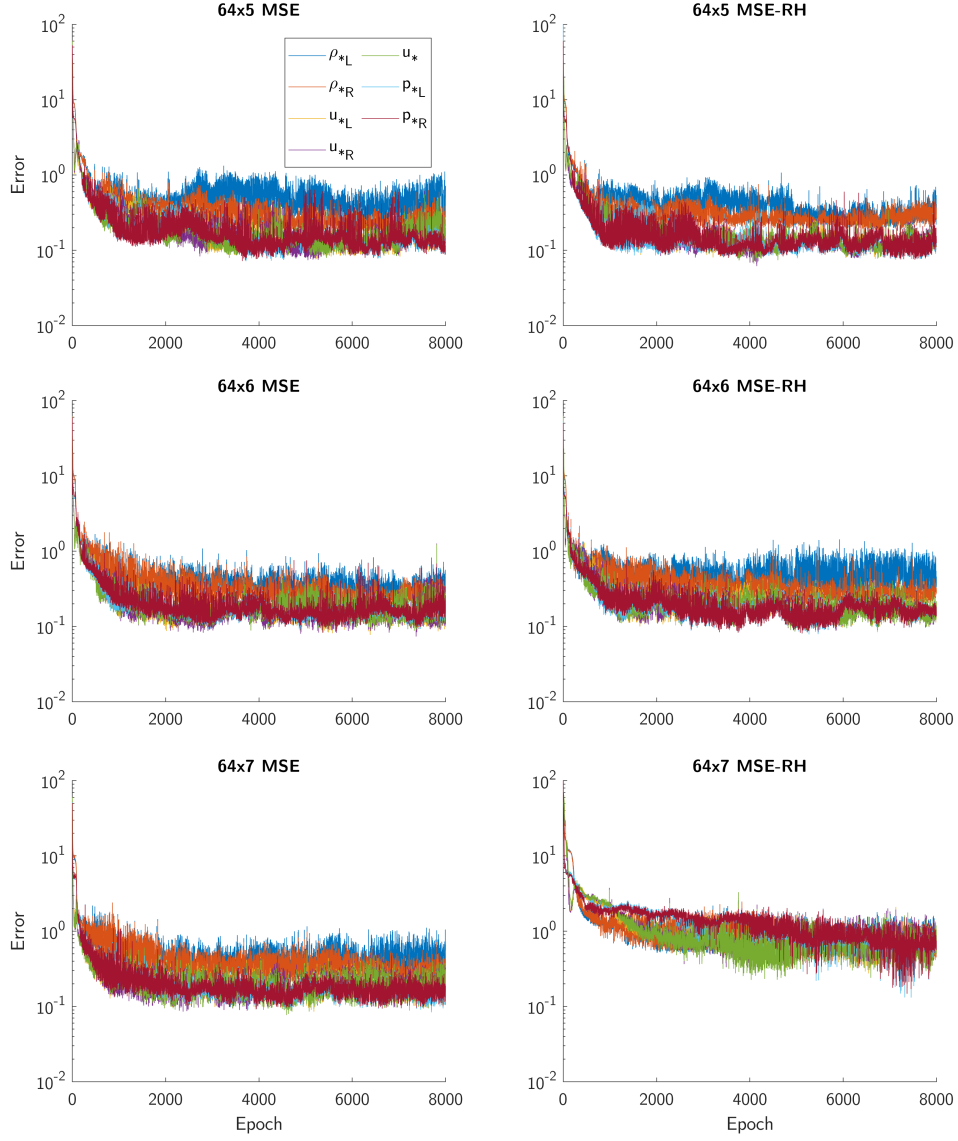


Figure 3: Learning curves showing mean relative L_1 test errors for all multi-layer perceptrons in Tab. 3. Network depth does not significantly affect the convergence rates nor the final errors and losses. All networks except 64×7 MSE-RH converged to errors $< 1\%$.

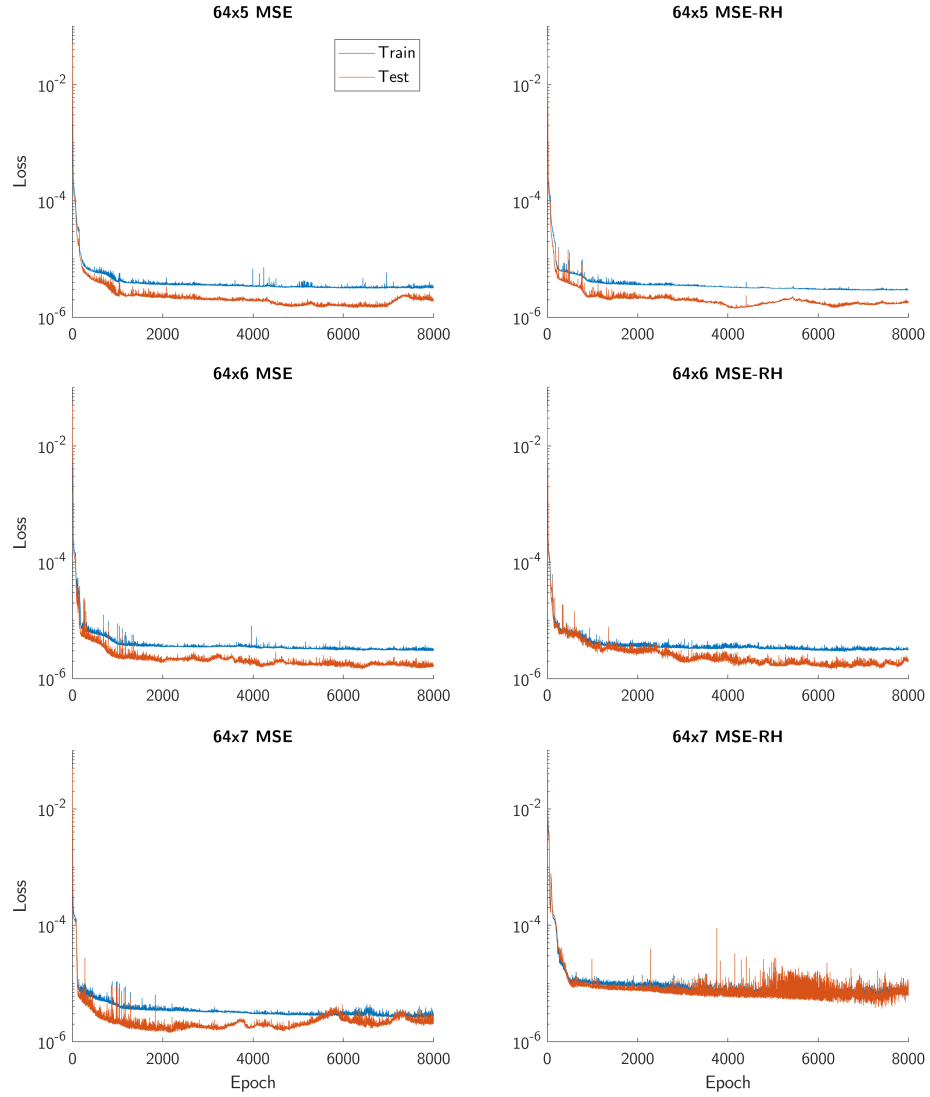


Figure 4: Learning curves showing train and test losses for all multi-layer perceptrons in Tab. 3. All 64×5 and 64×6 networks converged to similar losses, while the 64×7 MSE-RH network exhibited poor convergence. The lower test losses are likely caused by bias in the test sample, however final train and test errors were similar.

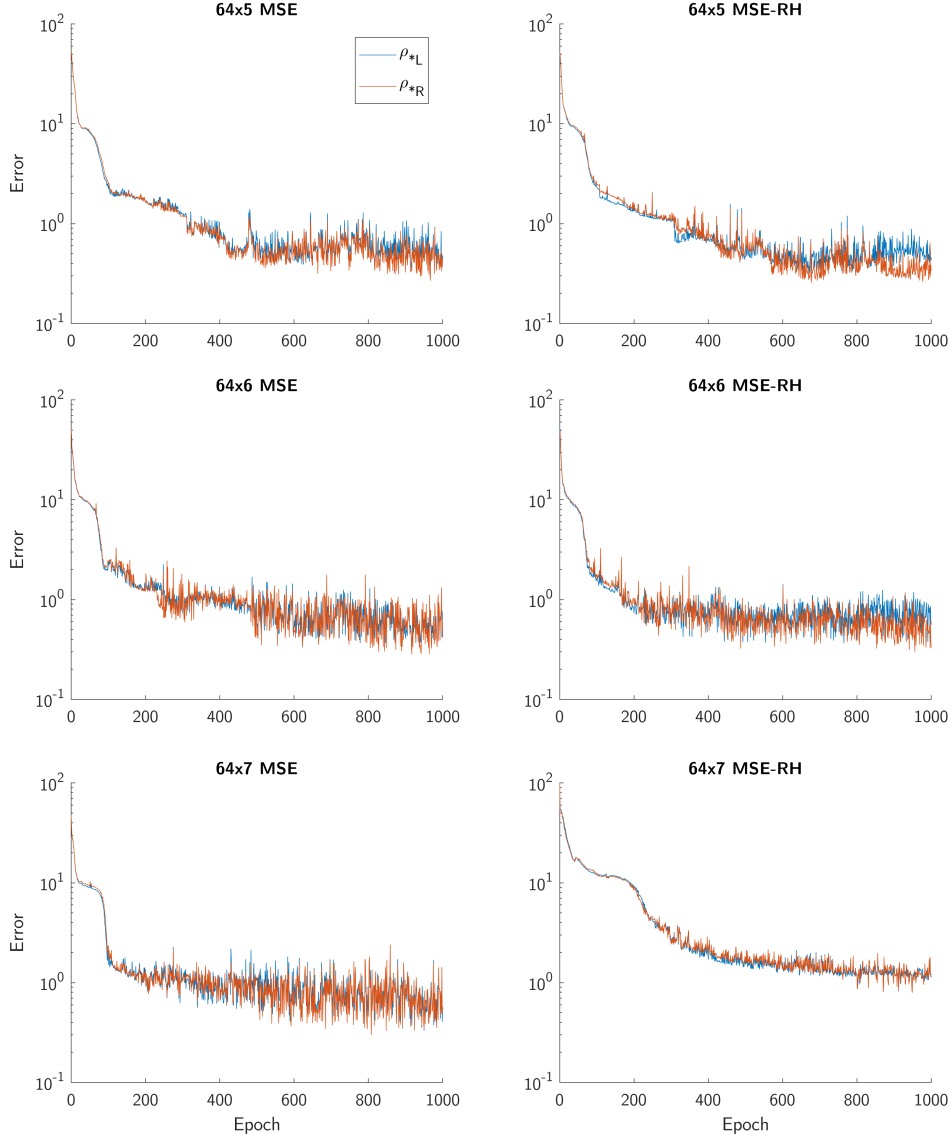


Figure 5: Learning curves truncated at 1000 epochs, showing test errors only for ρ_{*L}, ρ_{*R} across all multi-layer perceptrons trained from Tab. 3. Here it is visible that the MSE networks exhibit less stable convergence compared to the physics-informed MSE-RH networks.

The 64×7 MSE-RH network bears anomalous final test errors, losses, and training curves. Final train and test losses are nearly double that of the other FluxNets, and unlike all other FluxNets, the loss curves start to oscillate noticeably around 4000 epochs. It is worth noting that attempts to vary the learning rate still resulted in the same convergence behaviour and noisy loss curves. Taken together, these observations imply that the difference is related to both the number of weights and the Rankine-Hugoniot constraint in 64×7 MSE-RH. Neural networks become more prone to overfitting as the number of weights and number of training epochs increase. It is plausible that in 64×7 MSE-RH, the oscillations arise as the Rankine-Hugoniot constraint competes against overfitting tendencies due to greater number of weights. This would also explain why the same oscillations and poor performance are not observed in the 64×7 MSE network. The 64×7 MSE network is free to overfit, and indeed, it has the lowest train loss and greatest test loss of all networks examined. The overfitting explanation is further supported by the noisy numerical results in §7.2. This hypothetical limit on the number of weights to prevent overfitting is, however, also a function of the training data. Were a different range of the thermodynamic state space sampled in the training data, then this threshold would likely change as well. Different sampling techniques, such as a gradient-weighted approach that increases point density in regions of steep thermodynamic changes, could also shift the threshold at which overfitting becomes problematic.

In comparing the networks among each other, it is clear that the 64×5 MSE and MSE-RH networks are favourable for further evaluation, due to their simplicity yet similar performance to the other architectures. While the error plots in Fig. 3 show all curves spanning all training epochs and predicted variables, finer insights may be gleaned by considering only a subset of the curves. Fig. 5 shows the density learning curves for the first 1000 epochs. The error curves for the MSE-RH networks appear to oscillate less and at smaller amplitudes than the equivalent MSE networks, potentially due to the less naive, physics-informed search process. Except for the 64×7 MSE-RH network, all networks also approached $O(1)\%$ error around 200 epochs with both MSE and MSE-RH networks converging at roughly the same rate. Despite the apparent noise in mean relative L_1 errors, we can be satisfied that all networks converged to an acceptable degree given the decreasing, relatively noise-free trends in the loss curves, excepting 64×7 MSE-RH.

Neural networks thus provide an order of magnitude reduction in the

mean relative L_1 test errors compared to the Roe family of solvers, which ranged from 1 to 4% as shown in Tab. 4. Though this improvement may seem marginal, even minor inaccuracies in transcritical numerical simulations can destabilize solutions and lead to spurious results. In addition, only the mean test errors have been discussed. It is compelling to analyze the *distribution* of test errors across the entire dataset.

Fig. 6 contains box plots of the relative L_1 errors across the entire dataset including train and test samples. Data points are identified as outliers if they are greater than $q_e + w(q_3 - q_1)$ or less than $q_1 - w(q_3 - q_1)$ where w corresponds to 3σ , q_1 is the first quartile, and q_3 is the third quartile. The 64×5 MSE-RH FluxNet is evidently more precise in that its error distributions are significantly tighter across all star-state variables. Additionally, the mean errors of the 64×5 MSE-RH FluxNet are more closely aligned with zero, whereas the mean error of the Roe family of solvers overpredicts density and pressure by $\sim 2\%$, and underpredicts velocity by $\sim 1\%$. Among the plots for the 64×5 MSE-RH FluxNet, it is also worth addressing the greater number of outliers in ρ_{*L} and wider middle quartiles in ρ_{*R} as compared to in $u_{*L}, u_{*R}, p_{*L}, p_{*R}$ —this may be attributed to the nature of the Rankine-Hugoniot condition. Pressure and velocity are explicitly enforced as constant across the contact discontinuity, whereas density can vary and is implicitly enforced through the nonlinear wave structures on either side of the star-state region. The full set of implicit nonlinear physics that constrain density is more challenging to represent in the network, though the density error distributions continue to be tighter, less biased, and contain fewer outliers than with Roe-type solvers.

Given the sensitivity of transcritical flow simulations to small errors, it is also instructive to consider the maximum errors encountered across the entire dataset used in this study. Tab. 5 shows the relative L_∞ errors of 64×5 MSE-RH versus the Roe family of solvers, thus enabling an assessment of worst-case performance. It can be seen that the L_∞ errors of 64×5 MSE-RH are an order of magnitude smaller compared to the Roe errors. Nevertheless, the L_∞ errors of either solver are significant versus the mean L_1 errors discussed earlier. Future FluxNets may be able to reduce L_∞ errors further by penalizing large L_∞ errors encountered during the training process, or by avoiding the use of the given FluxNet for flow conditions where large errors may be more prevalent.

Last but not least, the error distribution may be analyzed through a probability density plot, such as Fig. 7 which plots the error probability

density function in ρ_{*L} by normalizing a histogram of errors against the total number of data points. Compared to 64×5 MSE-RH’s dense clustering around zero, the mode of the Roe distribution is at zero although the errors are more evenly spread out with a smaller peak at 2.5%. Interestingly, the 64×5 MSE-RH network shows a small accumulation of errors around 4%. This is due to certain data points which involved uniform or nearly uniform initial conditions where finite floating point precision was sufficient to introduce $O(10^{-2})$ error fluctuations in the the exact-iterative solver that produced the training data. Nevertheless, it was necessary to include these points in order for the networks to handle cases of uniform or nearly uniform initial conditions.

Table 5: Relative L_∞ error (% of the exact solution) in all star-state variables estimated by the 64×5 MSE-RH FluxNet versus the Roe family of solvers.

Solver	ρ_{*L}	u_{*L}	p_{*L}	u_*	ρ_{*R}	u_{*R}	p_{*R}
64×5 MSE-RH	57.3	2.4	13.3	2.8	4.5	2.6	13.3
Roe	132.6	41.5	144.2	41.5	232.9	41.5	144.2

7.2. Transcritical shock tube

The 64×5 MSE and 64×5 MSE-RH FluxNets were used to solve the transcritical shock tube test case studied in Ma *et al.* [59] and Wang & Hickey [31], leveraging the first-order Godunov scheme described in §2. The initial conditions for this test case are:

$$\begin{bmatrix} \rho_L \\ u_L \\ p_L \end{bmatrix} = \begin{bmatrix} 180 \text{ kg m}^{-3} \\ 150 \text{ m s}^{-1} \\ 11 \text{ MPa} \end{bmatrix}; \quad \begin{bmatrix} \rho_R \\ u_R \\ p_R \end{bmatrix} = \begin{bmatrix} 7.4 \text{ kg m}^{-3} \\ 50 \text{ m s}^{-1} \\ 0.2 \text{ MPa} \end{bmatrix} \quad (22)$$

The computational domain consists of 256 cells that are uniformly distributed over $x \in [-1, 1]$. A CFL number of 0.5, based on the acoustic time scale, is used. Transparent (*i.e.* transmissive, non-reflecting) boundary conditions are implemented via ghost cells, as described in Toro [22]. A grid convergence study was performed in an earlier study [31].

Unfortunately, the 64×5 MSE network caused significant numerical instabilities that produced negative temperatures and internal energies. To determine whether this would always be the case, the 64×5 MSE network was tested on a slightly modified version of the original problem initialized

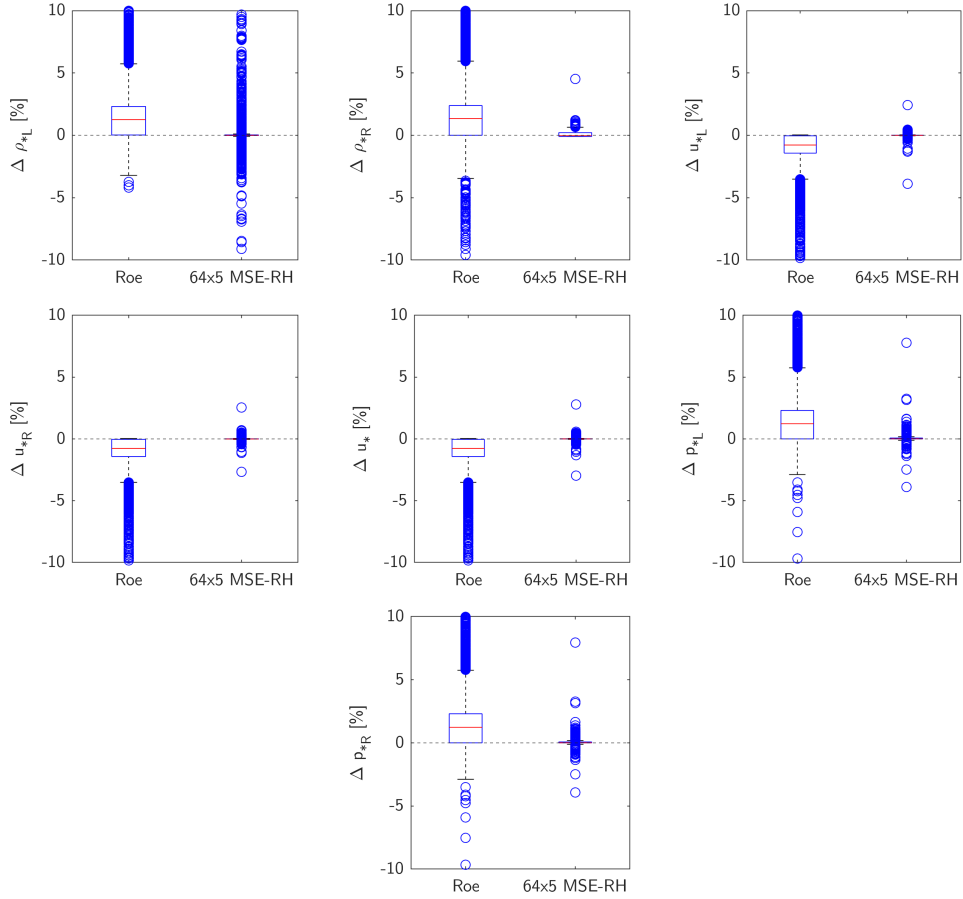


Figure 6: Box plots showing the distribution of relative L_1 errors in all star-state variables estimated by 64×5 MSE-RH FluxNet versus the Roe family of solvers. The red lines indicate the mean, the boxes enclose data from the 25th to the 75th percentile, and the circles indicate outliers. Vertical axes are rescaled to make boxes more visible—many outliers, especially for the Roe results, sit beyond the vertical limits. Errors were calculated across the entire dataset including train and test samples.

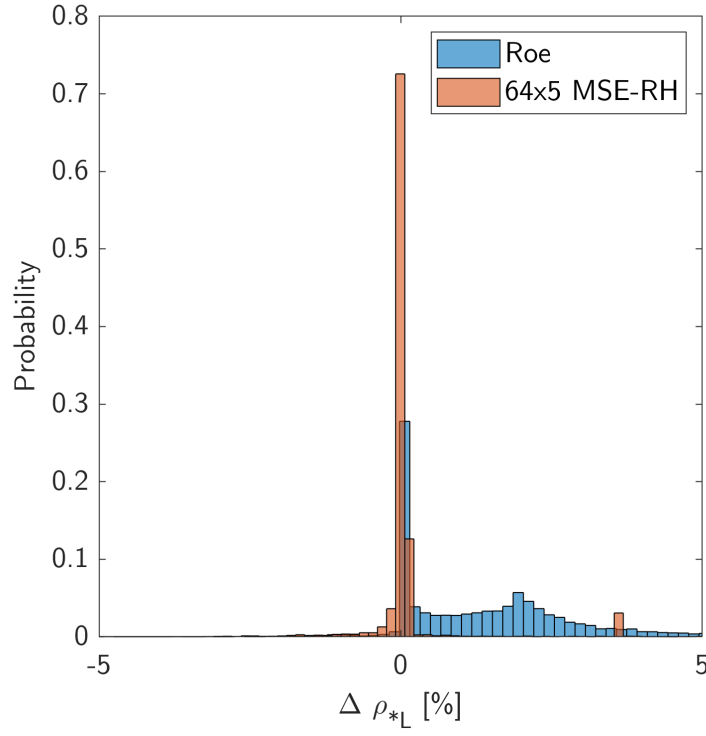


Figure 7: Probability of relative L_1 errors in ρ_{*L} for the 64×5 MSE-RH FluxNet versus the Roe family of solvers. The horizontal axis is rescaled to improve resolution near the mean—many outliers sit beyond the horizontal limits. Errors were calculated across the entire dataset including train and test samples.

to a lower left pressure, density, and velocity as well as slightly higher right pressure, density, and velocity. This change was designed to reduce the gradients during the initial time steps before the self-similar wave structure of the Riemann problem has stabilized. On the modified problem, 64×5 MSE did not produce negative thermodynamic variables (Fig. 8). Notwithstanding, the results are unacceptable. There are visible errors throughout the rarefaction wave region, as well as significant noise near the tail of the rarefaction where the wave meets the left star state. The region to the right of the shock is also unusually diffusive considering that information should not propagate upstream of a normal shock even in a first-order scheme. This would only be possible if the wavespeeds calculated from the star-state predictions by the FluxNet resulted in fluxes taken from an incorrect state of the Riemann problem. This may also explain why oscillations are most severe at interface between the rarefaction tail and left star state: the flux solver needs to accurately calculate wavespeeds in order to ensure entropy correctness and proper upwinding. Considering that the 64×5 MSE network had lower train losses and higher test losses than the 64×5 MSE-RH network during training (Tab. 4), this adds to the pool of evidence that the non-physics-informed network overfitted the data.

Table 6: Absolute L_1 errors at $x = 0$ for primitive variables ρ, u, p_r achieved by the Roe, Roe-StARS, and 64×5 MSE-RH solvers in the non-ideal transcritical shock tube of Fig. 9. Errors for Roe-Harten are repeated here from Wang & Hickey [31] for reference. Percentage reduction in the absolute Roe error is shown in parentheses. For reference, the exact solution is $[\rho, u, p_r] = [110.9 \text{ kg m}^{-3}, 276.3 \text{ m s}^{-1}, 1.511]$.

Error	Roe	Roe-Harten	Roe-StARS	64×5 MSE-RH
$\Delta \rho$	33.0	13.5 (−59.2%)	12.9 (−61.1%)	11.8 (−64.2%)
Δu	−56.2	−15.2 (−73.0%)	−13.8 (−75.4%)	−11.8 (−79.1%)
Δp_r	0.680	0.222 (−67.4%)	0.208 (−69.4%)	0.171 (−74.9%)

Meanwhile, 64×5 MSE-RH generated smooth and physically consistent solutions with the original transcritical shock tube. To facilitate comparison with the exact and Roe-StARS solutions computed earlier, Fig. 9 plots solutions to the original problem rather than the modified problem used for 64×5 MSE. Looking at $x = 0$ where Roe normally suffers from an expansion shock nonphysicality, it is immediately clear that 64×5 MSE-RH not only alleviates this error but is more smooth and accurate in this region than even the Roe-StARS solver (Tab. 6). Minor differences between the Roe-StARS and 64×5 MSE-RH solution are also observed throughout the solution. For

instance, 64×5 MSE-RH predicts a normal shock position slightly to the left of that predicted by Roe-StARS, closer to the exact solution.

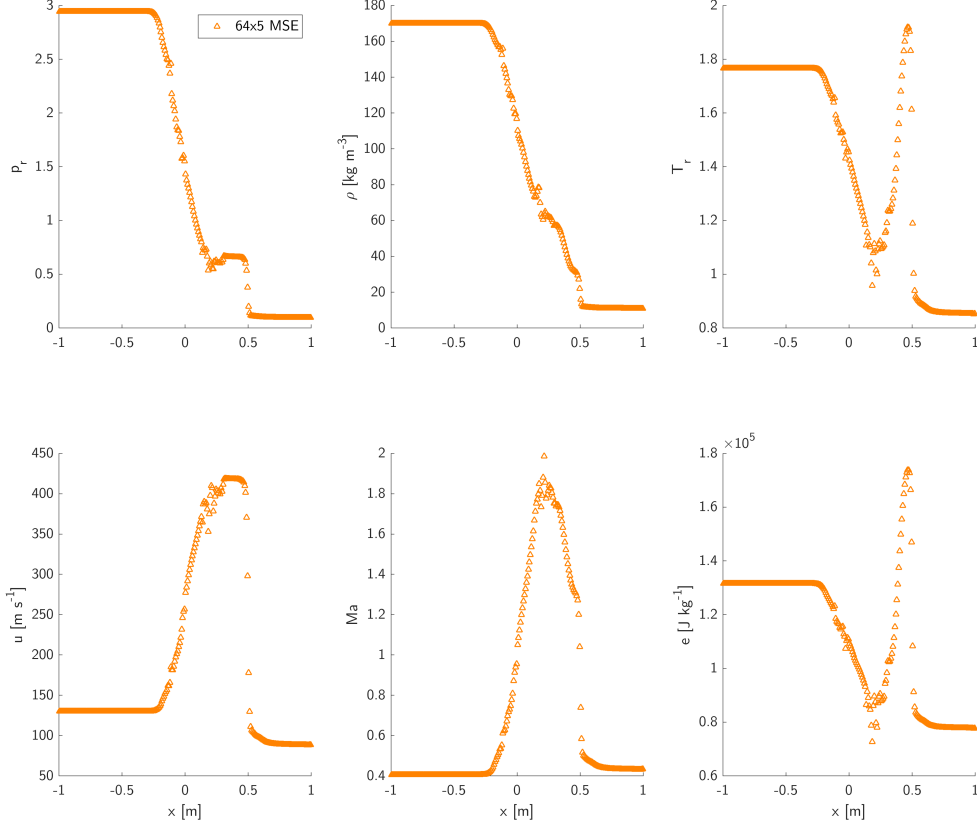


Figure 8: Numerical results for a transcritical nitrogen shock tube solved with the 64×5 MSE FluxNet, showing oscillatory and spurious solutions. Non-ideal thermodynamics and the Peng-Robinson equation of state [30] are applied.

7.3. Two-dimensional Riemann problem

In order to further evaluate the implications of a FluxNet approach, a 2D Riemann problem test case was solved with the 64×5 MSE-RH FluxNet and the Roe solver [42]. Similar to the 1D Riemann problem, the 2D problem serves as a test case for verifying numerical codes. Additionally, in higher dimensions, more complex wave interactions such as Mach diffraction and reflection can occur. More elaborate schemes and meshes may also be deployed

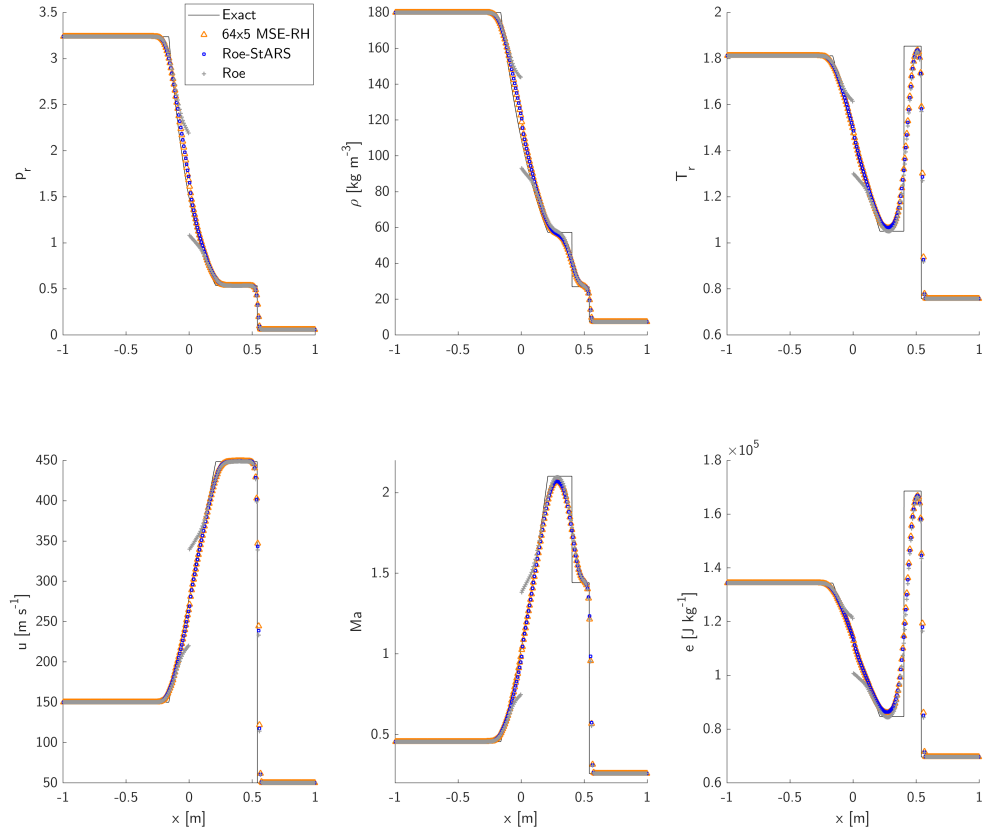


Figure 9: Numerical results for a transcritical nitrogen shock tube solved with non-ideal thermodynamics and the Peng-Robinson equation of state [30]. Results for the Roe, Roe-StARS, and 64×5 MSE-RH FluxNet solver are shown.

to improve accuracy and stability because waves propagate not only in orthogonal directions but at angles oblique to the unit basis vectors [22, 61]. However, for the present purposes, it suffices to consider a uniform Cartesian grid and just one of the potential configurations of a 2D Riemann problem—it is the the Riemann solver, and less so the scheme, that is under consideration.

Fig. 10 provides the initial conditions and expected wave evolution for a transcritical 2D Riemann problem inspired by the case studied in Wang & Hickey [31]. Here, u is the velocity in the x -direction and w is the velocity in the y -direction. Transparent boundary conditions were set at $x = -1m, 1m$ and $y = -1m, 1m$. A CFL number of 0.45 was used. The computational domain consisted of 256×256 cells. The numerical scheme was advanced until $t = 0.0009s$. The medium comprised transcritical nitrogen gas modelled with the Peng-Robinson state equation [30] and full non-ideal thermodynamics. Results for the Roe and 64×5 MSE-RH solvers are shown in Figs. 11 and 12. Analytical solutions are unavailable for the 2D Riemann problem, therefore the subsequent discussion is based on concepts developed for the 1D Riemann problem [22, 31].

The most obvious difference between the Roe and FluxNet results is that nonphysical shocks are greatly diminished in the FluxNet solution. In the Roe solution, expansion shock fronts manifest along the negative x and y axes, even extending slightly past the origin. These shocks remain stationary with respect to time, thus preventing information from propagating across these spurious boundaries. By contrast, the FluxNet solution shows a finer, more even distribution of characteristic waves that is typical of isentropic expansion fans. The nonphysical shock fronts also do not extend as far past the origin. The mitigation of expansion shocks further changes the nature of the peak that forms just to the upper right of the origin. In the FluxNet solution, the pressure, density, temperature, and internal energy are all higher in this region. This is in contrast to the Roe solution, where the artificial shocks produce relatively low post-shock conditions.

Finally, it is worth noting that the FluxNet solution contains small kinks in u along the $y = 0$ expansion shock and in w along the $x = 0$ expansion shock. These kinks do not appear in the Roe solution. This is because the FluxNet attempts to spread out the characteristic lines when it encounters an entropy violation, but the presence of an orthogonal shock front leads to a discontinuity and distortion in the characteristic lines along that front. This could be mitigated in the future by incorporating additional physical constraints or mesh design assumptions into the data preparation stage, so

that multidimensional considerations—such as flux splitting, averaging, or weighing—are encoded into the network training. However, the emphasis of this study was on the Riemann solver itself, and more advanced aspects of mesh design are left to future study with new datasets. Overall, the 2D test cases demonstrate mitigation of nonphysical flow features even when a physically constrained solver designed for 1D flow is applied to a relatively simple mesh.

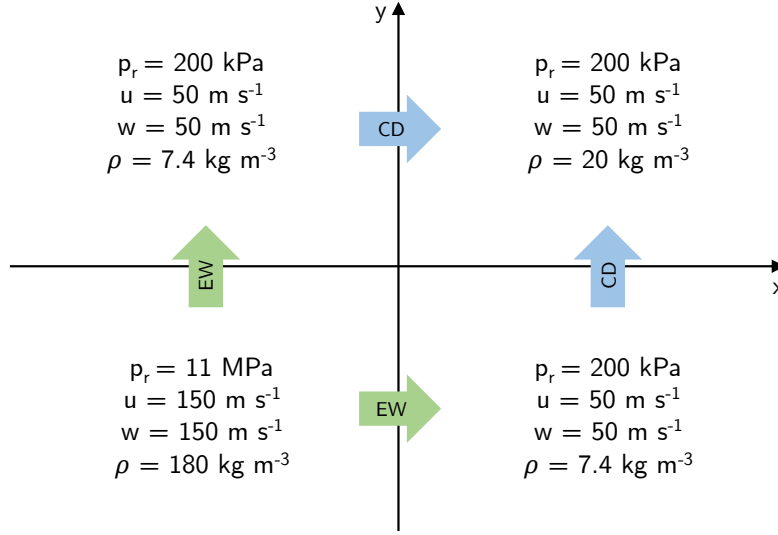


Figure 10: Initial conditions for the 2D Riemann problem test case considered in this study, based loosely on the case considered in Wang & Hickey [31]. The expected wave pattern is shown consisting of two contact discontinuities (*i.e.* slip lines in 2D) and two expansion waves.

7.4. Comparison of accuracy and time complexity

Sufficient data is now available to perform a critical comparison of the different types of Riemann solvers examined throughout this study. Entropy violation errors are summarized in Tab. 6, mean errors are contained in Tab. 4, and error distributions are plotted in Fig. 6. A time complexity table is also provided in Tab. 7.

As far as computational efficiency is concerned, it is manifest that any entropy-stable Riemann solver involves $O(10^2)$ greater time complexity than the unmodified Roe solver. Whereas Roe-Harten and Roe-StARS exhibit minor differences in time complexity between ideal and non-ideal gases, as

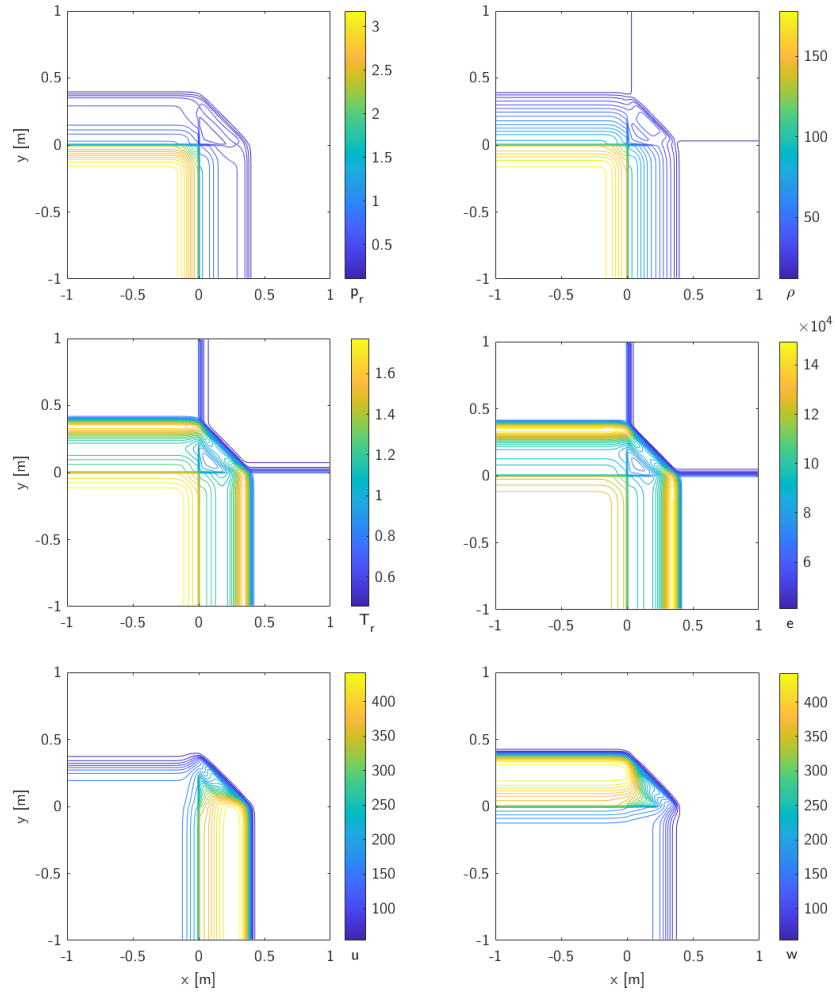


Figure 11: Solution to the 2D Riemann problem test case at $t = 0.0009s$, as computed with a Roe solver. Two expansion waves and two slip lines are visible, along with nonphysical expansion shocks along the negative x and y axes. e here is presented in units of J kg^{-1} .

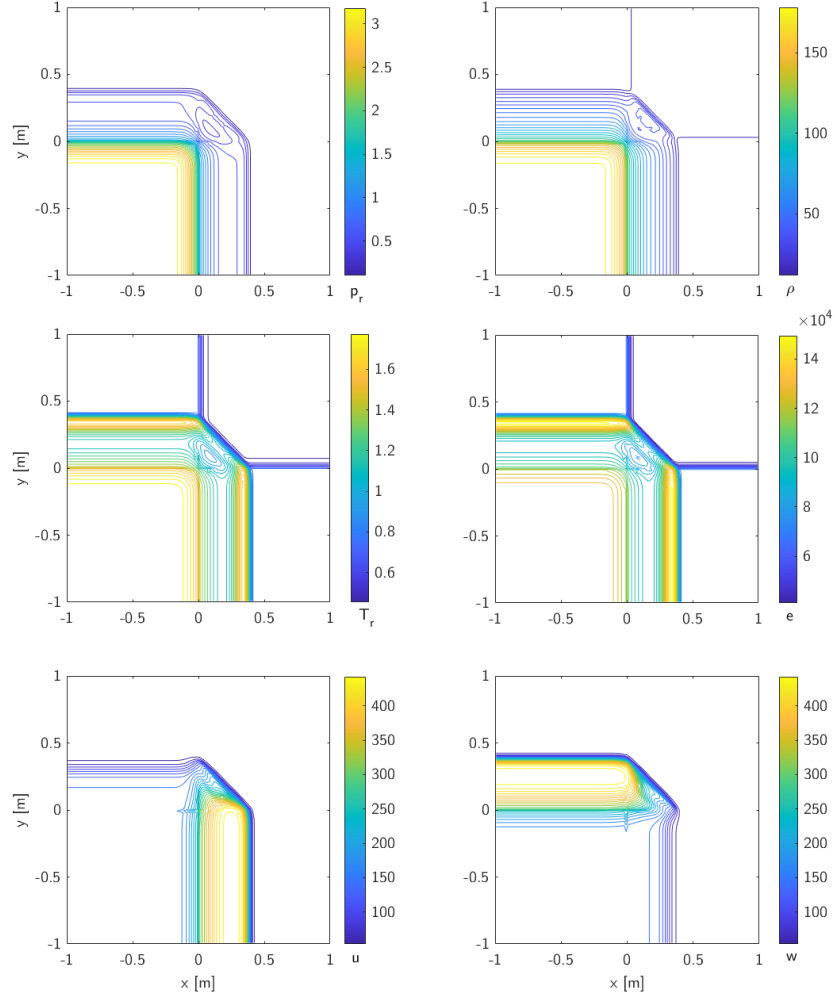


Figure 12: Solution to the 2D Riemann problem test case at $t = 0.0009s$, as computed with the 64×5 MSE-RH FluxNet. Two expansion waves and two slip lines are visible, and unlike the Roe solution, the FluxNet solutions significantly mitigate any nonphysicalities. e here is presented in units of $J kg^{-1}$.

Table 7: Time complexity required to solve the flux of a single cell interface using the Roe, Roe-StARS, 64×7 MSE-RH, or exact-iterative solvers. Time complexity for Roe-Harten is repeated here from Wang & Hickey [31] for reference.

Solver	No Entropy Violation		With Entropy Violation	
	Ideal Gas	Peng-Robinson Gas	Ideal Gas	Peng-Robinson Gas
Roe	123	157	123	157
Roe-Harten	229 (+86%)	331 (+111%)	239 (+94%)	341 (+117%)
Roe-StARS	229 (+86%)	331 (+111%)	285 (+132%)	387 (+146%)
64×5 MSE-RH	441 (+258%)	441 (+181%)	441 (+258%)	441 (+181%)
Exact	524 (+326%)	1799 (+1046%)	524 (+326%)	1799 (+1046%)

well as whether an entropy violation is present, the FluxNet approach is equally demanding under all circumstances. It is necessary to highlight as well that methods such as Roe-Harten require tuning the appropriate amount of artificial diffusivity to balance entropy stability with sharpness, whereas Roe-StARS needs no tuning and is thus more generic in its design and application. FluxNets, on the other hand, are indirectly the product of heuristic decisions such as the training data and hyperparameters that influence the learning process subject to physical constraints.

In terms of errors and accuracy, the Roe solver is likely inappropriate for many transcritical flows where entropy violations must be avoided. Roe-Harten, Roe-StARS, and FluxNets offer varying levels of entropy stability, mean error, and error distribution that are each beneficial for different flow problems. When transcritical flow problems span relatively small regions of the thermodynamic state space, the Roe-Harten or Roe-StARS solvers are most appropriate due their efficiency yet effective mitigation of entropy errors. Roe-StARS in particular avoids re-tuning and unnecessary diffusivity. However, when flow problems span relatively large thermodynamic variations and numerous time steps, a learning-based approach may be more sensible. The error distribution plots suggest that over a wide thermodynamic state space, traditional Roe-type solvers exhibit greater biases and a wider range of errors in star-state conditions compared to the physics-informed FluxNet. Such errors would naturally accumulate as simulation time progresses, as discussed by Wang & Hickey [31]. Lastly, the time required to prepare a suitable dataset and train a network cannot be overlooked. Using the earlier noted laptop, the exact solver required 21 hours to iterate through 64,000 Riemann problems—and only one-third of these points converged for the

thermodynamic state space considered in this study. Furthermore, training the 64×5 MSE-RH FluxNet required approximately 3 days. These times could be reduced in the future by taking advantage of parallelism and utilizing computing clusters. Although the exact amount of time required will vary depending on server specifications, this upfront investment on the order of one week is readily justified provided that the FluxNet is trained on a suitable dataset to be generally applicable without the need for repeated dataset regeneration and training.

8. Conclusion

In this study, a variety of learning-based Riemann solvers or *FluxNets* were conceptualized, trained, and evaluated on transcritical flow problems. Key findings were established related to network architecture, training strategies, and performance in terms of accuracy and time complexity.

Four types of inputs and output formulations were analyzed: conservative to conservative, primitive to conservative, all-primitives to all-primitives, and a primitive to primitive formulation excluding temperature. The preliminary feasibility assessments showed that the latter exhibited the most promising training behaviour, with other designs failing to converge. Additionally, it was observed that including a physics-informed energy constraint was unnecessary to achieve low errors and instead destabilized the training process.

Subsequently, different FluxNets with varying network size and loss function were trained and tested. It was shown that machine learning has the potential to eliminate entropy violations and reduce errors in general compared to the Roe solver, while achieving time complexity lower than exact solvers. Moreover, a physics-informed approach was demonstrated to be crucial to ensuring smooth, numerically stable, and generalizable results that are physically realistic. In the context of non-ideal thermodynamics, where traditional approximate Riemann solvers such as Roe can yield inaccurate results, the added computational cost of a learning-based approach may be worth the reductions in error. Future studies are advised to study mixtures, non-classical gases (*e.g.* BZT fluids), higher-order schemes, additional test cases, and the role of training data and sampling schemes in influencing the network size required to achieve a given level of accuracy.

Acknowledgments

We thank the Natural Sciences and Engineering Research Council of Canada (NSERC) and University of Waterloo Graduate Fellowships for funding this research.

Statement of Data Availability

The codes used in this study are available at <https://git.uwaterloo.ca/jc9wang/fluxnet>.

References

- [1] M. Ma, J. Lu, G. Tryggvason, Using statistical learning to close two-fluid multiphase flow equations for a simple bubbly system, *Physics of Fluids* 27 (9) (2015).
- [2] K. Duraisamy, G. Iaccarino, H. Xiao, Turbulence modeling in the age of data, *Annual Review of Fluid Mechanics* 51 (1) (2019) 357–377.
- [3] O. San, R. Maulik, M. Ahmed, An artificial neural network framework for reduced order modeling of transient flows, *Communications in Non-linear Science and Numerical Simulation* 77 (2019) 271–287.
- [4] S. Pawar, S. E. Ahmed, O. San, A. Rasheed, An evolve-then-correct reduced order model for hidden fluid dynamics, *Mathematics* 8 (570) (2020).
- [5] P. J. Milan, J.-P. Hickey, X. Wang, V. Yang, Deep-learning accelerated calculation of real-fluid properties in numerical simulation of complex flowfields, *Journal of Computational Physics* 444 (110567) (2021).
- [6] A. Maleki, A. Haghighi, I. Mahariq, Machine learning-based approaches for modeling thermophysical properties of hybrid nanofluids: a comprehensive review, *Journal of Molecular Liquids* 322 (114843) (2021).
- [7] M. Veit, S. K. Jain, S. Bonakala, I. Rudra, D. Hohl, G. Csanyi, Equation of state of fluid methane from first principles with machine learning potentials, *Journal of Chemical Theory and Computation* 15 (4) (2019) 2574–2586.

- [8] Y. Fujimoto, K. Fukushima, K. Murase, Methodology study of machine learning for the neutron star equation of state, *Physical Review D* 98 (023019) (2018).
- [9] K. Zhu, E. A. Mueller, Generating a machine-learned equation of state for fluid properties, *Journal of Physical Chemistry B* 124 (39) (2020) 8628–8639.
- [10] W. E. Faller, S. J. Schreck, Neural networks: applications and opportunities in aeronautics, *Progress in Aerospace Sciences* 32 (1996) 433–456.
- [11] C. Lee, J. Kim, D. Babcock, R. Goodman, Application of neural networks to turbulence control for drag reduction, *Physics of Fluids* 9 (1997) 1740–1747.
- [12] A. T. Mohan, D. V. Gaitonde, A deep learning based approach to reduced order modeling for turbulent flow control using lstm neural networks, *arXiv:Computational Physics* (2018).
- [13] F. Gueniat, L. Mathelin, M. Y. Hussaini, A statistical learning strategy for closed-loop control of fluid flows, *Theory of Computational Fluid Dynamics* 30 (2016) 497–510.
- [14] M. Dissanayake, N. Phan-Thien, Neural-network-based approximations for solving partial differential equations, *Communications in Numerical Methods in Engineering* 10 (1994) 195–201.
- [15] R. Gonzalez-Garcia, R. Rico-Martinez, I. Kevrekidis, Identification of distributed parameter systems: a neural net based approach, *Computers and Chemical Engineering* 22 (1998) 965–968.
- [16] I. E. Lagaris, A. Likas, D. I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Transactions on Neural Networks* 9 (1998) 987–1000.
- [17] B. Lusch, J. N. Kutz, S. L. Brunton, Deep learning for universal linear embeddings of nonlinear dynamics, *Nature Communications* 9 (1) (2018).
- [18] S. L. Brunton, B. R. Noack, P. Koumoutsakos, Machine learning for fluid mechanics, *Annual Review of Fluid Mechanics* 52 (2020) 477–508.

- [19] G. E. K. M. Raissi, P. Perdikaris, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational Physics* 378 (2019) 686–707.
- [20] G. E. K. Zhiping Mao, Ameya D. Jagtap, Physics-informed neural networks for high-speed flows, *Computer Methods in Applied Mechanics and Engineering* 360 (112789) (2020).
- [21] D. A. Bezgin, S. J. Schmidt, N. A. Adams, A data-driven physics-informed finite volume scheme for nonclassical undercompressive shocks, *Journal of Computational Physics* 437 (110324) (2021).
- [22] E. F. Toro, *Riemann Solvers and Numerical Methods for Fluid Dynamics*, 3rd Edition, Springer, 2009.
- [23] J. Magiera, D. Ray, J. S. Hesthaven, C. Rohde, Constraint-aware neural networks for riemannproblems, *Journal of Computational Physics* 409 (109345) (2019).
- [24] H. Hugoniot, Memoire sur la propagation des mouvements dans les corps et specialement dans les gaz parfaits, *Journal de l'Ecole Polytechnique* 58 (1887) 1–125.
- [25] V. Gyrya, M. J. Shashkov, A. N. Skurikhin, S. Tokareva, Machine learning approaches for the solution of the riemann problem in fluid dynamics: a case study (with reviewers), *Journal of Computational Physics* (2019).
- [26] O. Fuks, H. A. Tchelepi, Limitations of physics-informed machine learning for nonlinear two-phase transport in porous media, *Journal of Machine Learning for Modeling and Computing* 1 (1) (2022) 19–37.
- [27] T. Dieselhorst, W. Cook, S. Bernuzzi, D. Radice, Machine learning for conservative-to-primitive in relativistic hydrodynamics, *Symmetry* 13 (11) (2021).
- [28] J. C. H. Wang, Riemann solvers with non-ideal thermodynamics: exact, approximate, and machine learning solutions, Phd thesis, University of Waterloo (2022).

- [29] J. C. H. Wang, J.-P. Hickey, Fluxnet: A physics-informed learning-based riemann solver for transcritical flows with non-ideal thermodynamics, Preprint (2022).
- [30] D.-Y. Peng, D. B. Robinson, A new two-constant equation of state, *Industrial & Engineering Chemistry Fundamentals* 15 (1) (1976) 59–64.
- [31] J. C. H. Wang, J.-P. Hickey, A class of structurally complete approximate riemann solvers for trans- and supercritical flows with large gradients, *Journal of Computational Physics* 468 (111521) (2022).
- [32] B. E. Poling, J. M. Prausnitz, J. P. O’Connell, *The Properties of Gases and Liquids*, McGraw-Hill, 2001.
- [33] C. L. Yaws, *Chemical Properties Handbook: Physical, Thermodynamic, Environmental, Transport, Safety, and Health Related Properties for Organic and Inorganic Chemicals*, McGraw-Hill, 1999.
- [34] J. R. Andrews, O. Biblarz, Temperature dependence of gases in polynomial form, *Naval Postgraduate School* (1981).
- [35] J. C.-H. Wang, J.-P. Hickey, Analytical solutions to shock and expansion waves for non-ideal equations of state, *Physics of Fluids* 32 (2020) 086105.
- [36] S. K. Godunov, Finite difference method for numerical computation of discontinuous solutions of the equations of fluid dynamics, *Matematicheskii Sbornik* 47 (89-3) (1959).
- [37] B. Riemann, Ueber die fortpflanzung ebener luftwellen von endlicher schwingungsweite, *Abhandlungen der Koeniglichen Gesellschaft der Wissenschaften zu Goettingen* 8 (1860) 43–66.
- [38] R. Menikoff, B. J. Plohr, The riemann problem for fluid flow of real gases, *Reviews of Modern Physics* 61 (1) (1989) 75–130.
- [39] R. Saurel, M. Larini, J. C. Loraud, Exact and approximate riemann solvers for real gases, *Journal of Computational Physics* 112 (1994) 126–127.

- [40] P. Colella, H. M. Glaz, Efficient solution algorithms for the riemann problem for real gases, *Journal of Computational Physics* 59 (1985) 264–289.
- [41] J. W. Banks, On exact conservation for the euler equations with complex equations of state, *Communications in Computational Physics* 8 (5) (2010) 995–1015.
- [42] P. L. Roe, Approximate riemann solvers, parameter vectors, and difference schemes, *Journal of Computational Physics* 43 (1981) 357–372.
- [43] A. Harten, High resolution schemes for hyperbolic conservation laws, *Journal of Computational Physics* 49 (1983) 357–393.
- [44] E. F. Toro, A linearised riemann solver for the time-dependent euler equations of gas dynamics, *Proceedings of the Royal Society of London* 434 (1991) 683–693.
- [45] S. Gottlieb, C.-W. Shu, E. Tadmor, Strong stability-preserving high-order time discretization methods, *SIAM Review* 43 (1) (2001) 89–112.
- [46] T. M. Mitchell, *Machine Learning*, McGraw-Hill, 1997.
- [47] S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd Edition, Prentice Hall, 2010.
- [48] M. Ihme, C. Schmitt, H. Pitsch, Optimal artificial neural networks and tabulation methods for chemistry representation in les of a bluff-body swirl-stabilized flame, *Proceedings of the Combustion Institute* (2009).
- [49] S. Bhalla, M. Yao, J.-P. Hickey, M. Crowley, Compact representation of a multi-dimensional combustion manifold using deep neural networks, *European Conference on Machine Learning* (2019).
- [50] P. J. Milan, X. Wang, J.-P. Hickey, Y. Li, V. Yang, Accelerating numerical simulations of supercritical fluid flows using deep neural networks, *AIAA ScieTech Forum* (2020).
- [51] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.
- [52] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2nd Edition, Springer, 2017.

- [53] G. Hinton, N. Srivastava, K. Swersky, Neural networks for machine learning: Lecture 6, Coursera (2012).
- [54] D. P. Kingma, J. Ba, Adam: a method for stochastic optimization, in: International Conference on Learning Representations, 2015.
- [55] G. Cybenko, Approximation by superpositions of a sigmoidal function, *Mathematics of Controls, Signals, and Systems* 2 (4) (1989) 303–314.
- [56] K. Hornik, Approximation capabilities of multilayer feedforward networks, *Neural Networks* 4 (2) (1991) 251–257.
- [57] M. Leshno, V. Y. Lin, A. Pinkus, S. Schocken, Multilayer feedforward networks with a nonpolynomial activation function can approximate any function, *Neural Networks* 6 (6) (1993) 861–867.
- [58] E. Dekel, D. Nassimi, S. Sahni, Parallel matrix and graph algorithms, *SIAM Journal on Computing* 10 (4) (1981) 657–675.
- [59] P. C. Ma, Y. Lv, M. Ihme, An entropy-stable hybrid scheme for simulations of transcritical real-fluid flows, *Journal of Computational Physics* 340 (2017) 330–357.
- [60] D. T. Banuti, M. Raju, P. C. Ma, M. Ihme, J.-P. Hickey, Seven questions about supercritical fluids towards a new fluid state diagram, *AIAA SciTech Forum* (2017).
- [61] R. Leveque, *Finite-Volume Methods for Hyperbolic Problems*, Cambridge University Press, 2002.